

PROJET DE FIN D'ETUDES

Présenté pour l'obtention du titre :

Master Data Science for Smart Manufacturing

Par : MOUDDEN Hamza

Titre :

Community Detection in Social
Networks for Targeted Marketing and
Recommender Systems

Jury :

Pr . FRIKH Bouchra	Présidente	EST –USMBA –Fès
Pr . BOUGTEB Yahya	Rapporteur	FP – USMBA –Taza
Pr . AKACHAR Elyazid	Encadrant	FS – UMI – Meknès

Acknowledgements

First and foremost, I thank Almighty God for granting me the strength, patience, and determination necessary to complete this work.

This final year's project would not have been possible without the support and valuable contribution of several people, to whom I wish to express my deep gratitude.

I would like to warmly thank my supervisor, Professor ELYAZID Akachar, for agreeing to guide this work. I am extremely grateful for his availability, insightful advice, patience, and the trust he placed in me throughout this journey. His relevant guidance and scientific rigor have greatly contributed to the improvement of this project and to my personal development.

My thanks also go to the jury members for taking the time to evaluate this work. Their remarks and suggestions will be valuable in enriching this research and opening new perspectives.

I wish to express my gratitude to all the professors of the master's program in Data Science for Smart Manufacturing. Their dedication, the quality of their teaching, and their support throughout my training have been decisive in the success of my academic journey.

A great thank you to my parents, for their unconditional love, their sacrifices, and their unwavering support. Their belief in me has been my driving force during difficult times. This project is the result of their continuous support and encouragement.

I do not forget my friends and fellow classmates, with whom I shared unforgettable moments, enriching discussions, and valuable mutual support. Their friendship has made this journey more enjoyable and stimulating.

Abstract

The detection of communities in social networks has attracted sustained research interest, yet most existing studies evaluate algorithms solely on structural metrics such as modularity, without assessing their downstream utility for real-world applications. This project addresses that gap by conducting a dual-phase experimental study that first benchmarks community detection performance and then measures the direct impact of detected communities on recommender system quality.

In the first phase, thirteen classical algorithms including Louvain, Leiden, Infomap, and FluidC and four deep learning methods GCN, GAT, VGAE, and Autoencoder were evaluated across nine real-world benchmark datasets and synthetic LFR networks ranging from 500 to 100,000 nodes. Results show that classical modularity-optimizing algorithms consistently outperform deep learning approaches in both structural quality and computational efficiency (in large graphs $+500$), with Louvain and Leiden achieving near-perfect NMI scores on well-structured networks and scaling effectively to large graphs.

In the second phase, the four best-performing algorithms were integrated into hybrid recommendation pipelines combining community partitions with latent factor models (SVD, ALS) and lexical scoring (BM25), evaluated on the UCI Online Retail dataset. The hybrid ALS+Louvain approach, after hyperparameter tuning, achieved an $F1@5$ score of 0.3709 a 58% improvement over the pure community baseline demonstrating that community structure and latent factors capture complementary aspects of user preferences. A deployable Streamlit web application was also developed to make the full pipeline interactive and accessible without code modification.

These findings suggest that classical community detection remains the most practical choice for ecommerce recommendation at scale, and that the choice of downstream scoring model and its hyperparameters are at least as important as the community detection algorithm itself.

Keywords: community detection, social networks, recommender systems, targeted marketing, Louvain, Leiden, GCN, collaborative filtering, ALS, SVD, BM25.

La détection de communautés dans les réseaux sociaux a suscité un intérêt de recherche soutenu, mais la plupart des études existantes évaluent les algorithmes uniquement sur des métriques structurelles telles que la modularité, sans mesurer leur utilité réelle pour des applications concrètes. Ce projet comble cette lacune en menant une étude expérimentale en deux phases : la première évalue les performances de détection de communautés, et la seconde mesure l'impact direct des communautés détectées sur la qualité des systèmes de recommandation.

Dans la première phase, treize algorithmes classiques dont Louvain, Leiden, Infomap et FluidC et quatre méthodes d'apprentissage profond GCN, GAT, VGAE et Autoencoder ont été évalués sur neuf jeux de données de référence réels et des réseaux LFR synthétiques allant de 500 à 100 000 nœuds. Les résultats montrent que les algorithmes classiques d'optimisation de la modularité surpassent systématiquement les approches d'apprentissage profond en termes de qualité structurelle et d'efficacité computationnelle, Louvain et Leiden atteignant des scores NMI quasi parfaits sur les réseaux bien structurés.

Dans la seconde phase, les quatre algorithmes les plus performants ont été intégrés dans des pipelines de recommandation hybrides combinant les partitions communautaires avec des modèles à facteurs latents (SVD, ALS) et un scoring lexical (BM25), évalués sur le jeu de données UCI Online Retail. L'approche hybride ALS+Louvain, après optimisation des hyperparamètres, a atteint un score $F1@5$ de 0,3709 soit une amélioration de 58 % par rapport à la référence purement communautaire démontrant que la structure communautaire et les facteurs latents capturent des aspects complémentaires des préférences utilisateurs. Une application web Streamlit a également été développée pour rendre l'ensemble du pipeline interactif et accessible sans modification de code.

Ces résultats suggèrent que la détection de communautés classique reste le choix le plus pratique pour la recommandation en e-commerce à grande échelle, et que le choix du modèle de scoring en aval ainsi que ses hyperparamètres sont au moins aussi importants que l'algorithme de détection de communautés lui-même.

Mots-clés : détection de communautés, réseaux sociaux, systèmes de recommandation, marketing ciblé, Louvain, Leiden, GCN, filtrage collaboratif, ALS, SVD, BM25.

يعد اكتشاف المجتمعات في الشبكات الاجتماعية من المجالات التي شهدت اهتمامًا كبيرًا خلال السنوات الأخيرة، إلا أن أغلب الدراسات ركزت على مقارنة خوارزميات اكتشاف المجتمعات بالاعتماد على مؤشرات تقييم مثل Modularity، دون التحقق من مدى مساهمة هذه الخوارزميات في تحسين التطبيقات العملية، خاصة أنظمة التوصية. لذلك يهدف هذا المشروع إلى دراسة أداء خوارزميات اكتشاف المجتمعات، ثم تقييم أثرها عند دمجها داخل نظام توصية .

اعتمدت الدراسة على مرحلتين. في المرحلة الأولى تمت مقارنة ثلاثة عشر خوارزمية تقليدية، من بينها Louvain و Leiden و FluidC و Infomap، بالإضافة إلى أربع خوارزميات تعتمد على التعلم العميق هي GCN و GAT و VGAE و Autoencoder. أُجريت التجارب على مجموعة من قواعد البيانات الحقيقية وشبكات LFR الاصطناعية التي يتراوح حجمها بين 500 و 100000 عقدة. وقد بينت النتائج أن الخوارزميات التقليدية، وخاصة Louvain و Leiden، قدمت أفضل النتائج من حيث جودة اكتشاف المجتمعات وسرعة التنفيذ، كما كانت أكثر ملاءمة لمعالجة الشبكات كبيرة الحجم مقارنة بخوارزميات التعلم العميق .

أما في المرحلة الثانية، فقد تم اختيار أفضل الخوارزميات ودمجها مع نماذج توصية مختلفة مثل ALS و SVD و BM25، ثم اختبارها باستعمال بيانات UCI Online Retail. وأظهرت النتائج أن النموذج الهجين ALS مع Louvain حقق أفضل أداء بعد ضبط المعاملات، حيث سجل قيمة $F1@5$ بلغت 0.3709، أي بزيادة تقارب 58% مقارنة بالنموذج الذي يعتمد على المجتمعات فقط. ويبين ذلك أن دمج معلومات المجتمعات مع نماذج الترشيح التعاوني يساعد على تقديم توصيات أكثر دقة .

وفي نهاية المشروع تم تطوير تطبيق باستخدام Streamlit يتيح تنفيذ مختلف مراحل الدراسة بطريقة تفاعلية.

وتبين من خلال هذه الدراسة أن الخوارزميات التقليدية ما زالت تمثل الحل الأنسب لاكتشاف المجتمعات في الشبكات الكبيرة، كما أن أداء نظام التوصية لا يعتمد على خوارزمية اكتشاف المجتمعات وحدها، بل يتأثر أيضًا باختيار نموذج التوصية وطريقة ضبط معاملاته .

الكلمات المفتاحية: اكتشاف المجتمعات، الشبكات الاجتماعية، أنظمة التوصية، التسويق الموجّه، Louvain، Leiden، GCN، الترشيح التعاوني، ALS، SVD، BM25.

Table of Contents

Chapter 1: General Introduction and Problem Statement	14
I. General Introduction	14
II. Background	15
III. Problem Statement	16
IV. Motivation	16
V. Objectives	17
VI. Structure of the Report	18
VII. Conclusion of Chapter	19
Chapter 2: State of the Art	21
I. Introduction to Chapter	21
II. Literature Review Methodology	21
III. Graph Theory Fundamentals	23
1. Generality	23
2. Directed and Undirected Graphs	23
3. Weighted Graphs	24
4. Adjacency Matrix	25
5. Degree and Centrality Measures	25
6. Paths and Connected Components	26
7. Graph Density	27
IV. Community Detection in Graphs	27
1. Definition of a Community	27
2. Local and Global Definitions of Communities	28
3. Graph Partitioning Perspective	29
4. Disjoint and Overlapping Communities	29
5. Quality Functions	30
V. Classical Community Detection Algorithms	33
1. Modularity-Based Algorithms	33
2. Random Walk-Based Algorithms	35
3. Statistical Physics-based Methods	37
4. Divisive Algorithms	37
5. Label Propagation Methods	39
6. Spectral Methods	41

7. Seed-centric Approaches.....	42
VI. Modern Approaches	42
1. Graph Embeddings	43
2. Deep Learning on Graphs	45
3. Advantages and Limitations of Community Detection Algorithms	49
VII. Community Detection Applications	51
1. General Application Domains.....	51
2. Community Detection for Targeted Marketing and E-Commerce	53
3. Integrating Textual Semantics	53
4. Community Detection for Recommender Systems.....	54
VIII. Conclusion of Chapter	55
Chapter 3: Community Detection for Recommender Systems	57
I. Introduction to Chapter	57
II. Foundations of Recommender Systems	57
1. Definition and Importance	57
2. Classical Paradigms	58
3. Challenges in Recommender Systems	59
4. Social Recommender Systems.....	59
III. Community Detection for Recommender Systems	60
1. Collaborative Filtering Enhanced by Community Structure.....	60
2. Addressing the Cold-Start Problem with Community Detection	61
3. Combining Semantic Content and Community Structure	61
4. Community-Based Collaborative Filtering	62
5. Community-Aware Matrix Factorization	63
IV. Proposed Hybrid Approaches	64
1. User-Based Approach.....	65
2. Item-Based Approach	66
3. Singular Value Decomposition with Community Filtering	67
4. Alternating Least Squares with Community Filtering	68
5. Best Matching 25 with Community Filtering	69
V. Conclusion of Chapter	71
Chapter 4: Implementation and Experimental Results	73
I. Introduction to Chapter	73
II. Experimental Setup	73

1. Hardware and Software Environment	73
2. Libraries and Frameworks	74
3. Standard Benchmark Datasets for Community Detection	74
4. Online Retail Dataset (For Recommender System Evaluation)	78
5. Evaluation Metrics	79
6. LFR Synthetic Benchmark Parameters	80
III. Experiment 1: Community Detection Results	80
1. Introduction	80
2. Results of Classical Algorithms	80
3. Results of Deep Learning Methods	82
4. LFR Benchmark Results	83
IV. Experiment 2: Recommender System Integration	87
1. Comparison of Recommender Systems.....	87
V. Experiment 3: Web Application Interface	89
1. Configuring the Algorithm Pipeline.....	90
2. Generating User-Based Recommendations	90
3. Generating Item-Based Recommendations	91
4. Community Trending Notifications	91
VI. Conclusion of Chapter	92
Chapter 5: Conclusion and Future Perspectives	94
I. Summary of Contributions.....	94
1. Answer to the Research Question.....	94
II. Future Perspectives	95
1. Dynamic Community Detection for Temporal Recommender Systems	95
2. Neural Collaborative Filtering with Community Embeddings	96
3. Cross-Domain Evaluation	96
4. Scalable Deep Learning for Large Graphs	96
5. Explainability and Trust	96

Table of Figures

Figure 1 : PRISMA 2020 flow diagram of the literature selection process.	22
Figure 2 : Undirected vs Directed Graphs.....	24
Figure 3 : Weighted Graphs	24
Figure 4 : Adjacency Matrix.....	25
Figure 5 : Betweenness concept	26
Figure 6 : Connected components of an undirected graph.....	26
Figure 7 : Dense vs Sparse Graphs.....	27
Figure 8 : Illustration of Community Structure in a graph.	27
Figure 9 : Balanced vs Real World Partition.....	29
Figure 10 : Disjoint and Overlapping communities	29
Figure 11 : Louvain Algorithm	33
Figure 12 : Leiden Algorithm	34
Figure 13 : Girvan Newman Algorithm	37
Figure 14 : LPA Algorithm	39
Figure 15 : Illustration of the graph embedding pipeline from knowledge graph to feature vectors.	43
Figure 16 : Architecture of a Multi-layer Graph Neural Network.	45
Figure 17 : Schematic representation of GCN layers.	46
Figure 18 : Graph Attention Network	47
Figure 19 : Architecture of a VGAE with encoder and decoder	48
Figure 20 : Social Network Analysis.....	51
Figure 21 : Biological Network Analysis	52
Figure 22 : Semantic-topological community detection framework	53
Figure 23 : Original vs Coarsened Graphs	54
Figure 24 : Overview of the CBMF recommendation pipeline.....	60
Figure 25 : Schematic representation of the hybrid food recommendation model [34].	62
Figure 26 : Comparison between traditional neighborhood-based collaborative filtering (a) and community-based collaborative filtering (b) [50].	63
Figure 27 : Community-constrained SVD matrix factorization workflow [49].	64
Figure 28 : User-Based Collaborative Filtering	65
Figure 29 : Item-Based Collaborative Filtering	66
Figure 30 : SVD of a matrix.....	67
Figure 31 : Alternating Least Squares Method	68
Figure 32 : Best Matching 25 Algorithm.....	69
Figure 33 : Zachary's Karate Club.....	75
Figure 34 : Impact of the parameter μ on the detectability of clusters in heterogeneous networks.	78
Figure 35 : LFR Benchmark Results: Network Size = 500 nodes	84
Figure 36 : LFR Benchmark Results: Network Size = 1000 nodes	84
Figure 37 : LFR Benchmark Results: Network Size = 5000 nodes.....	85
Figure 38 : LFR Benchmark Results: mixing parameter = 0.7	85
Figure 39 : LFR Benchmark Results: Network Size = 10000 nodes	86
Figure 40 : LFR Benchmark Results: Network Size = 50000 nodes	86
Figure 41 : LFR Benchmark Results: Network Size = 100000 nodes.....	87
Figure 42 : Hybrid Recall@5 scores vs community baselines.	88
Figure 43 : Precision-recall scatter plot illustrating trade-offs across different method categories.....	88
Figure 44 : Hybrid F1@5 scores vs community baselines.	89
Figure 45 : Pipeline Configuration	90
Figure 46 : User-Based Interface.....	90
Figure 47 : Item-Based Interface	91
Figure 48 : Trending Community Updates	92

List of Tables

Table 1 : Hardware and Software Environment.....	73
Table 2 : Libraries and Frameworks	74
Table 3 : LFR Synthetic Benchmark Parameters	80
Table 4 : NMI (Normalized Mutual Information) - Classical Algorithms	80
Table 5 : ARI (Adjusted Rand Index) - Classical Algorithms	81
Table 6 : Modularity - Classical Algorithms	82
Table 7 : Normalized Mutual Information (NMI) - Deep Learning Methods	82
Table 8 : Adjusted Rand Index (ARI) - Deep Learning Methods	83
Table 9 : Modularity - Deep Learning Methods.....	83
Table 10 : Hyperparameter Tuning Results for ALS and BM25	87

List of Acronyms

AE	Autoencoder	ALS	Alternating Least Squares
ARI	Adjusted Rand Index	BFS	Breadth-First Sampling
BM25	Best Matching 25	CBMF	Community-Based Matrix Factorization
CF	Collaborative Filtering	CLPA	Constrained Label Propagation Algorithm
Cluster-GCN	Clustering-based Graph Convolutional Network	DANE	Deep Attributed Network Embedding
DFS	Depth-First Sampling	DL	Deep Learning
DMGI	Deep Multiplex Graph Infomax	DRN	Deep Reconstruction Network
EBC	Edge Betweenness Centrality	F1	F1 Score
FANMF	Factorized Asymmetric Non-negative Matrix Factorization	FLPA	Fast Label Propagation Algorithm
FluidC	Fluid Communities Algorithm	GAN	Generative Adversarial Network
GAT	Graph Attention Network	GCN	Graph Convolutional Network
GGCRF	General Graph Coarsening Recommendation Framework	GN	Girvan–Newman Algorithm
GNN	Graph Neural Network	Goodreads	Goodreads Book Reviews Dataset
GraphMAE	Graph Masked Autoencoder	GraphSAGE	Graph SAmple and aggreGatE
GraphSAINT	Graph Sampling and Aggregation of Inductive Node Training	HDMI	High-order Deep Multiplex Infomax
KL	Kernighan-Lin Algorithm	LastFM	Last.fm Social Network
LDA	Latent Dirichlet Allocation	LFM	Local Fitness Maximization
LFR	Lancichinetti-Fortunato-Radicchi	LightGCN	Light Graph Convolutional Network
LPA	Label Propagation Algorithm	LPA-MNI	LPA with Modularity and Node Importance

MAE	Mean Absolute Error	MAGNN	Metapath Aggregated Graph Neural Network
MCL	Markov Clustering	MemLPA	Memory Label Propagation Algorithm
MF	Matrix Factorization	MGAE	Marginalized Graph AutoEncoder
MovieLens	MovieLens Rating Dataset	NDCG	Normalized Discounted Cumulative Gain
NGCF	Neural Graph Collaborative Filtering	NMF	Non-Negative Matrix Factorization
NMI	Normalized Mutual Information	PCA	Principal Component Analysis
Polbooks	Political Books Network	PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
RFID	Radio-Frequency Identification	RI	Rand Index
RMSE	Root Mean Square Error	RS	Recommender System
SGD	Stochastic Gradient Descent	SimCMR	Similarity-Based Community Merge/Refinement
SNA	Social Network Analysis	SRS	Social Recommender System
SVD	Singular Value Decomposition	Thiers	Primary School Contact Network
UCI	University of California Irvine	VGAE	Variational Graph Autoencoder
VS13	Versus Benchmark Dataset 13	VS15	Versus Benchmark Dataset 15

Chapter 1: General Introduction and Problem Statement

Chapter 1: General Introduction and Problem Statement

I. General Introduction

The scale of social media today is difficult to overstate. With more than two billion people using Facebook by 2018, and rival platforms generating hundreds of millions of daily posts and comments, the scale of modern social media is unprecedented. The collected data includes various forms of user activity, such as liking posts, sharing content, leaving comments, and following other users. When examined together, these actions form intricate networks that reveal important patterns about how people connect and interact with one another.

This clustering behavior is not merely a sociological curiosity. From a computational standpoint, it creates a structure that neither raw user attributes nor simple popularity metrics can fully capture. One particularly important type of structure is the presence of groups where members are more strongly connected to each other than to people outside the group. These clusters are commonly referred to as communities in network science. The task of automatically identifying these groups is known as community detection, and it represents a fundamental challenge in the analysis of complex networks.

This project applies community detection techniques to graphs derived from social media, with two practical purposes in mind. The first is targeted marketing delivering advertisements to specific groups of users likely to be interested in a product or service. The second is recommender systems, which suggest items, content, or connections that a user may find appealing.

By identifying communities of users who share common interests or behavioral patterns, marketers can move away from broadcasting the same message to everyone. Instead, they can tailor their campaigns to resonate with specific communities. Similarly, recommendation algorithms can take advantage of community information to provide more personalized suggestions. For example, a user might receive recommendations based on what other members of their community have enjoyed.

Combining community detection with recommendation is not straightforward. The first difficulty is algorithmic: dozens of methods exist, each optimizing a different objective. The second is evaluative: structural metrics like modularity do not necessarily predict recommendation quality. The third is practical: a method that works on a 100-node benchmark may collapse on a network with 100,000 users. This project addresses all three: measuring the quality of the communities that are found and effectively incorporating community information into real-world systems.

II. Background

Social media platforms can be modelled as graphs, where users are represented as nodes and interactions between them are represented as edges. These graphs are typically very large and sparse, meaning that only a small fraction of possible connections exist.

Social networks tend to exhibit two structural properties that directly affect how community detection behaves. First, average path lengths are surprisingly short even in massive networks a phenomenon Watts and Strogatz formalized as the small-world property. Second, degree distributions follow a power law: a small number of highly connected users coexist with a large majority who have few connections. These properties mean that standard clustering assumptions, such as uniform density or equal community size, are rarely held.

Community detection algorithms aim to divide the network into groups such that connections within the same group are dense, while connections between groups are limited. These communities often reflect real-world groupings such as shared interests or social circles.

This concept has practical importance. In marketing, identifying communities helps in targeting specific audiences and identifying influential users. In recommendation systems, it can improve personalization and help overcome issues like lack of data for new users or limited interaction history.

III. Problem Statement

Despite the large number of community detection algorithms available in the literature, selecting the most appropriate method for real-world applications remains a challenging task.

- Existing approaches differ significantly in their underlying principles, ranging from modularity optimization and random walks to embedding-based and deep learning methods. This diversity makes it difficult to determine which techniques are best suited for specific use cases.
- Most studies evaluate these algorithms using structural metrics such as modularity, which measure how well a network is partitioned. However, these metrics do not necessarily reflect the practical usefulness of the detected communities in applications such as marketing or recommendation systems.
- While modern approaches like graph embedding and graph neural networks have shown promising results, their added value compared to classical methods is not always clearly demonstrated, especially in large-scale social network scenarios.
- There is a lack of comparative analysis that considers both traditional and modern techniques within a unified framework, particularly when evaluated on realistic datasets and application-oriented criteria.

Based on these limitations, the main research question of this project is:

How do different community detection approaches, ranging from classical algorithms to modern graph learning methods, compare in terms of their effectiveness and practical utility for recommender systems?

IV. Motivation

When we started thinking about this research, one question kept coming back to me: does grouping people in a social network actually help us figure out what they want to buy or watch? Community detection has been studied for years as a graph problem, but honestly, there isn't much work that clearly shows whether these detected groups are useful for anything practical like making recommendations.

This led me to ask a few key questions. Are people in the same community truly alike in their purchasing or rating behavior? If a user is new and we have almost no data about them, can we use their community membership as a fallback to still give decent recommendations? With so many algorithms out there from simple ones like Louvain that run fast, to heavy deep learning models like GNNs which produce communities that help with recommendations? And finally, what's the best way to use this community information inside a recommender system?

Now, if we look at real e-commerce data, most of the user-item matrix is empty, over 98% of entries are missing [49], [50]. The problem is that traditional collaborative filtering needs users to have rated similar items to find reliable neighbors [36], and that just doesn't happen in practice. New users are even worse off: they have zero history, so there's nothing to base recommendations on [18]. Adding to the confusion, there are so many community detection algorithms available that it's genuinely hard to know which one to pick for a real project. Should we go with something fast and battle-tested like Louvain, or invest time and compute resources into training a graph neural network?

What we are aiming for with this work is to run a proper comparison and actually test how different algorithms perform when we plug them into a recommendation pipeline. we want to move beyond just looking at structural scores like modularity and see what really works in practice. At the end of the day, my goal is to figure out if community detection can become a genuinely useful tool for building smarter, more interpretable recommender systems not just an academic exercise.

V. Objectives

The main objectives of this project can be summarized as follows:

- To study and categorize community detection approaches, including both classical algorithms and modern graph-based learning methods
- To implement a selection of representative algorithms from different categories (e.g., Louvain, Label Propagation, and embedding-based methods)
- To apply these algorithms to real-world and synthetic network datasets
- To evaluate the detected communities using structural metrics such as modularity and conductance
- To assess the usefulness of these communities in practical applications, particularly targeted marketing and recommendation systems
- To compare the performance of different approaches and identify their strengths and limitations
- To provide insights into the trade-offs between simplicity, scalability, and effectiveness

VI. Structure of the Report

This report is organized into five chapters.

- Chapter 1 introduces the research context, defines the problem, explains the motivation behind this work, and states the main objectives. It also outlines the structure of the report.
- Chapter 2 reviews the state of the art in community detection. It covers graph theory fundamentals, classical algorithms, modern deep learning approaches, and discusses their advantages and limitations. It also presents the main application domains where community detection has proven useful.
- Chapter 3 narrows the focus to community detection for recommender systems. It covers the fundamentals of recommendation, reviews existing work at the intersection of the two fields, and presents the hybrid approaches we propose in this work.
- Chapter 4 presents the complete implementation and experimental results. It describes the experimental setup, evaluates community detection algorithms on real-world and synthetic benchmarks, and tests the proposed hybrid recommender approaches.
- Chapter 5 concludes the report with a summary of contributions, answers the main research question, and discusses future perspectives.
- Finally, a list of references and appendices are included at the end.

VII. Conclusion of Chapter

In this first chapter, we introduced the main ideas behind my research and explained why we think this topic is worth exploring. Social media platforms are huge now, and they generate a lot of data that can reveal a lot about how people behave and what they like. We also showed how we can turn these platforms into graphs where users become nodes and their interactions become edges which gives us a way to apply network analysis tools.

The main thing we wanted to get across is the community detection problem: how do we automatically find groups of users who are closely connected? And more importantly, can we actually use these groups for something useful, like making better recommendations? That's what led me to my main research question: how do different community detection methods compare when we look at how useful they actually are in recommender systems?

The reason we are doing this research is that even though there are many algorithms available, nobody really knows which ones work well for recommendations. If you work with e-commerce data, you quickly realize the user-item matrix is mostly empty, new users have no history to base anything on, and honestly, picking an algorithm is overwhelming. So, my goal is to make a solid comparison and figure out what helps in practice.

Now that we have set up the problem, the next chapter goes into literature. We will cover the basics of graph theory, look at the main community detection algorithms, both classic and modern, and see what other researchers have done with community information in recommender systems. This will give me the foundation we need to design my experiments in Chapter 4.

Chapter 2: State of the Art

Chapter 2: State of the Art

I. Introduction to Chapter

This chapter provides a comprehensive review of the literature on community detection. We'll start by explaining the methodology we used to identify and select the relevant studies for this review. Then we'll cover the fundamental concepts from graph theory that are essential for understanding community detection things like graph types, centrality measures, and network properties. After that, we'll define what communities are and discuss the challenges of evaluating their quality. The main part of the chapter surveys the various algorithms that have been proposed over the years, starting with classical approaches and moving to more recent deep learning methods. We'll also discuss the advantages and limitations of each family of algorithms based on what we found in the literature. Finally, we'll look at the main application domains where community detection has proven useful, which sets the context for why this research focuses on recommender systems.

II. Literature Review Methodology

The review process followed the PRISMA 2020 guidelines [53]. Three complementary identification strategies were combined to build the literature corpus: an initial set of seven articles recommended directly by the supervisor based on their direct relevance to the project's core themes, a structured database search targeting community detection and recommender system literature, and a supplementary identification phase relying on reference list screening, forward citation tracking, and technical documentation review.

The structured search was conducted on IEEE Xplore using the following Boolean string: ("community detection" OR "community structure" OR "graph clustering" OR "network clustering") AND ("recommender systems" OR "collaborative filtering" OR "recommendation" OR "recommender") AND ("social networks" OR "complex networks" OR "network science" OR "graph"). The search was restricted to English-language journal articles and reviews published between 2003 and 2026, yielding 409 records (292 conference papers, 107 journal articles, 4 magazine articles, and 3 book chapters). All seven supervisor-recommended articles also satisfied this search string, since each contains the term "community detection" in its title or abstract, confirming the relevance and discoverability of the supervisor's initial recommendations within the broader literature.

In addition to the database search, sixteen further records were identified through complementary methods, including snowballing techniques such as reference list screening and forward citation tracking [54], as well as direct review of technical documentation for software libraries and datasets used in the implementation, namely Scikit-learn's TruncatedSVD module, the implicit library for collaborative filtering, and the UCI Online Retail dataset.

Following deduplication, which removed 12 duplicate records, 413 records were screened by title and abstract, of which 351 were excluded for lack of relevance to community detection or recommender systems. The remaining 62 reports were sought for full-text retrieval; five could not be retrieved, leaving 57 reports assessed for eligibility. At this stage, five additional reports were excluded following full-text review for insufficient relevance to the research scope. Two reviewers independently screened titles and abstracts, with disagreements resolved through consensus. This process resulted in a final corpus of 52 studies, comprising 36 records identified through the database search (including the seven supervisor-recommended articles) and 16 records identified through supplementary methods, as summarized in Figure 1.

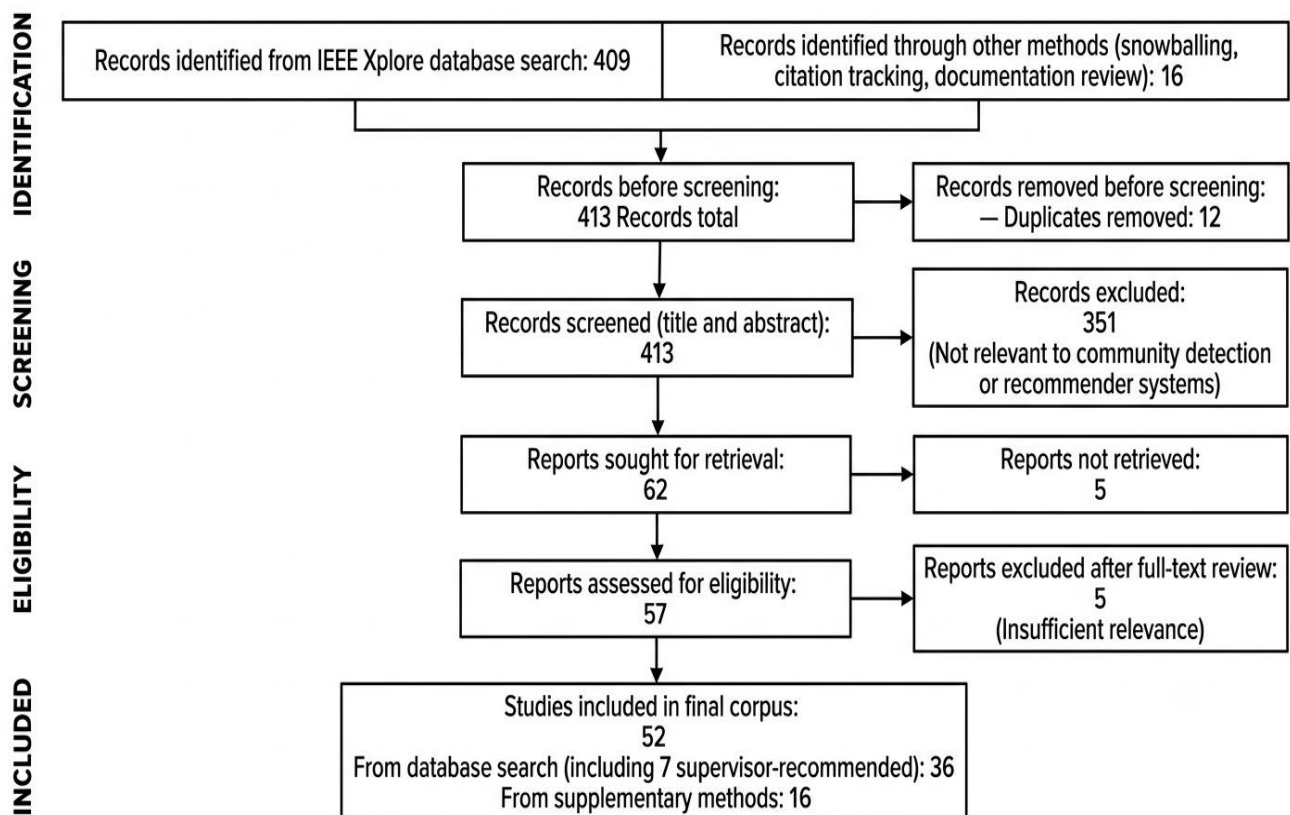


Figure 1 : PRISMA 2020 flow diagram of the literature selection process.

III. Graph Theory Fundamentals

1. Generality

Graph theory provides the mathematical framework used to model and analyze complex systems such as social networks, biological systems, and information networks. Networks from diverse domains, ranging from online social platforms to biological pathways, can be effectively represented using this formalism. In social media analysis, nodes typically represent individual users while edges capture various forms of interaction such as follows, likes, comments, or message exchanges.

In this context, a graph constitutes a framework formed by a collection of nodes, or vertices, joined through edges denoting pairwise associations among elements. This intuitive mapping from real-world users and their connections to a mathematical structure allows researchers to apply powerful analytical techniques to study user behavior and community formation.

Formally, a graph is denoted as $G = (V, E)$, where V is the set of nodes and $E \subseteq V \times V$ is the set of edges. This simple yet powerful abstraction enables the application of a wide range of analytical tools to study connectivity patterns and relational dynamics. Network analysis uses graph-based models due to their capacity for encoding structural layouts and systemic interaction patterns simultaneously [1].

2. Directed and Undirected Graphs

Graphs usually fall into two categories based on the kind of relationships they show: undirected or directed.

In undirected graphs, edges have no direction, so the relationship is symmetric. For instance, if you're modelling a friendship where both people feel the same way, an undirected graph works perfectly.

Directed graphs or digraphs, introduce orientation to every connection. Under this framework, a link originating at u toward v does not imply a reciprocal link. For example: following a user rarely guarantees mutual follows.

One big advantage of directed graphs is that they let you track incoming links (in-degree) and outgoing links (out-degree) separately, which tells you a lot more about each node's role.

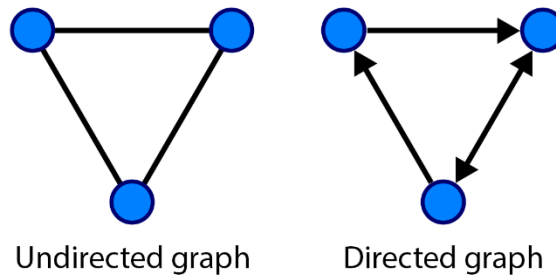


Figure 2 : Undirected vs Directed Graphs

3. Weighted Graphs

In a lot of real-world situations, not every connection between nodes carries the same meaning. That's where weighted graphs come in, they're basically an extension that adds a numerical value, or weight, to each edge.

That weight usually represents something like strength, frequency, or significance of interaction. Take a social network, for instance: you might use weights to capture the number of messages a group of people exchange, the number of likes or comments between them, or how often they interact.

Compared to unweighted graphs, weighted ones give you a much richer picture. They let algorithms consider not just whether a connection exists, but how strong it is [2].

The trade-off, though, is that adding weight makes things more computationally demanding. You also must be careful about normalizing the values depending on what algorithm you're using.

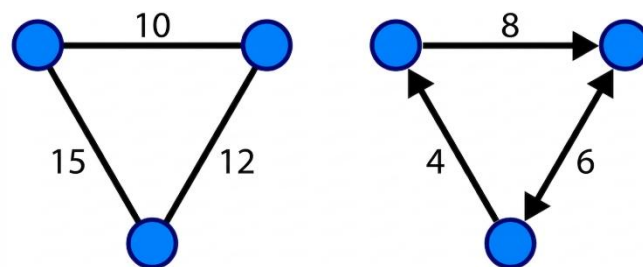


Figure 3 : Weighted Graphs

4. Adjacency Matrix

The adjacency matrix is a go-to mathematical method to represent graphs. For a graph with n nodes, it's an $n \times n$ matrix A where each A_{ij} gives information about the connection between node i and node j .

In an unweighted graph:

- $A_{ij} = 1$ if an edge exists between i and j
- $A_{ij} = 0$ otherwise

For weighted graphs, A_{ij} simply stores the weight of that edge.

Undirected graphs give you a symmetric adjacency matrix, whereas directed ones usually don't.

This representation shows up all over the place in practice especially in algorithms like spectral clustering, graph convolutional networks (GCNs), and a wide range of network analysis tasks.

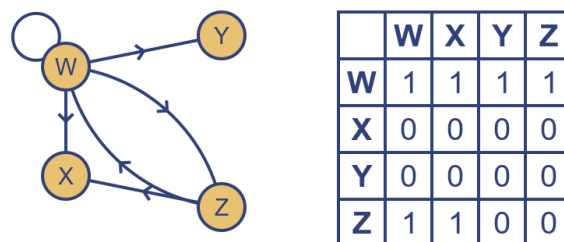


Figure 4 : Adjacency Matrix

5. Degree and Centrality Measures

One of the simplest but most powerful graph measures is node degree. It's just the number of edges connected to a node.

In undirected graphs, you get a single degree value. In directed graphs, there's a split: in-degree for incoming connections and out-degree for outgoing ones.

High-degree nodes are often seen as important or influential.

But if you want a deeper look at node importance, there are a few go-to centrality measures:

- Degree Centrality – basically just the degree itself; measures direct connections.

- Betweenness Centrality – tracks how often a node is visited on shortest paths between others. High value means it's acting as a bridge.
- Closeness Centrality – measures how fast a node can reach everyone else in the network.

These are especially useful for things like finding influencers, modeling how information spreads, and designing marketing decisions [3].

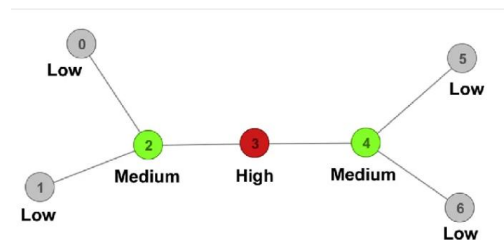


Figure 5 : Betweenness concept

6. Paths and Connected Components

In graph terms, a path is just a sequence of nodes where each pair in order is connected by an edge. Paths are essential for understanding how things travel through a network.

When there's a path between every pair of nodes, the graph is connected. If not, it splits into connected components subgraphs where nodes are all reachable from one another but not to nodes in other components.

Take real-world networks like social platforms: you mostly have one big, connected component, having most of the users, plus a handful of smaller isolated components.

Connectivity isn't just a theoretical concept, it's key for community detection, analyzing how resilient a network is, and modeling how information propagates.

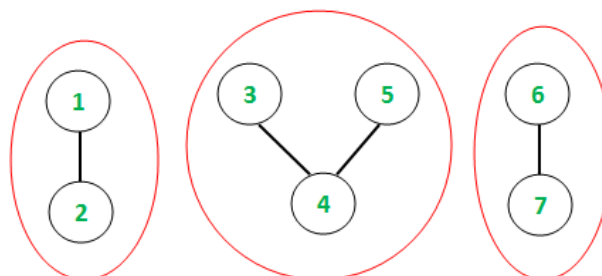


Figure 6 : Connected components of an undirected graph

7. Graph Density

Graph density is a simple way to describe how many edges a graph has relative to the maximum possible.

For undirected graphs, you calculate it as:

$$\text{Density} = \frac{2 | E |}{| V | (| V | - 1)}$$

Dense graphs have lots of edges; sparse graphs have relatively few.

Most real-world networks, especially social networks, are sparse. with each node connecting to only a small fraction of the other nodes.

This sparsity isn't just an observation it has real implications. It reduces computational costs and makes community detection more practical and meaningful.

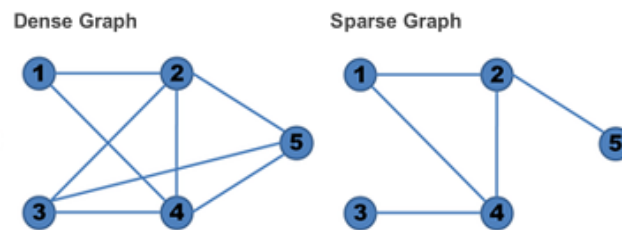


Figure 7 : Dense vs Sparse Graphs

IV. Community Detection in Graphs

1. Definition of a Community

Community is a central concept in network science, but there's no universally accepted definition.

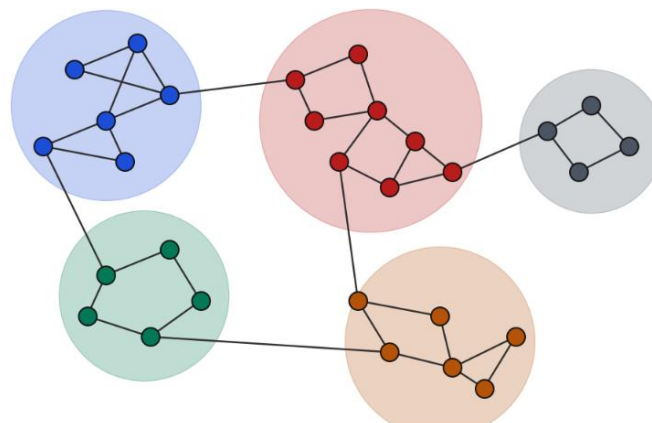


Figure 8 : Illustration of Community Structure in a graph.

As shown in Figure 8, Fortunato describes it as a group of nodes that are more densely connected internally than externally [4]. This definition is deliberately loose. A karate club that splits into two factions and a set of co-expressed genes in a biological network are both communities under Fortunato's definition, yet the structure driving each is entirely different. This ambiguity is not a flaw in the definition it reflects a genuine property of the problem.

Fortunato notes that this ambiguity comes from the diversity of real-world networks. What counts as dense or significant varies, so different algorithms often give different results on the same network [4]. Later research reinforces this, framing community detection as inherently ill-posed and context-dependent [5], [6].

Newer approaches have pushed beyond structural definitions. Probabilistic models define communities by shared connection patterns, while embedding-based methods locate communities in latent spaces where structurally similar nodes sit closer together [7], [8]. Together, these reflect a broader evolution from purely graph-theory definitions toward more data-driven representations.

2. Local and Global Definitions of Communities

Community definitions generally split into local and global approaches.

Local definitions focus on subgraph structure. A set of nodes counts as a community if it meets local properties such as high internal density or a strong clustering coefficient. These methods are fast computationally but can overlook large-scale network organization [4].

Global definitions consider the whole network and optimize an overall objective. Modularity is a prime example: it compares edge density inside communities to what you'd see in a random graph [4], [9].

According to Fortunato, global methods tend to give more consistent, meaningful partitions, but they come with heavy computational costs [4]. Recent advances address this through scalable approximations and parallel algorithms [10].

Modern approaches also incorporate machine learning, letting us infer communities from both structure and node attributes. These hybrid methods offer more flexible and expressive community representations [7], [11].

3. Graph Partitioning Perspective

Community detection can also be framed as a graph partitioning problem: divide the network so that connections inside each group are maximized and connections between groups are minimized [4].

Traditional graph partitioning comes with strict constraints like requiring a predefined number of clusters and forcing them to be balanced in size.

But real-world networks rarely fit that mold. Social networks, for example, have skewed degree distributions and highly uneven community sizes, so classical methods often fall short [5].

Modern approaches address this by supporting:

- Flexible community sizes
- Hierarchical structures
- More adaptable partitioning criteria

These advances have made community detection far more applicable to real-world scenarios [6], [10].

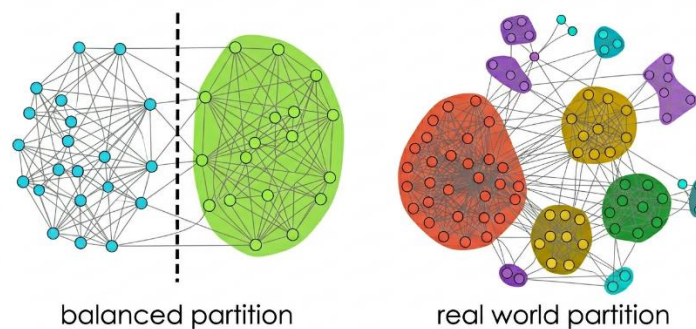


Figure 9 : Balanced vs Real World Partition

4. Disjoint and Overlapping Communities

Community structures can also be categorized by how membership works.

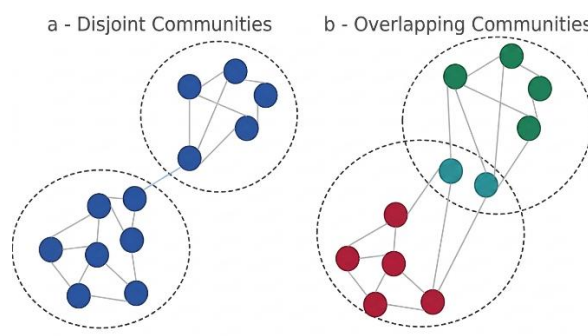


Figure 10 : Disjoint and Overlapping communities

In disjoint communities, each node belongs to exactly one group. This keeps things simple and is common in classical algorithms, but it often oversimplifies real-world complexity.

Overlapping communities allow nodes to belong to multiple groups at once. That's especially useful for social networks, where people are part of different circles like family, work, or hobbies [5], [12].

Fortunato discusses overlapping detection techniques such as clique percolation, which finds communities by linking fully connected subgraphs into chains [4].

Studies show that overlapping community detection leads to more realistic representations and better results in areas like recommender systems and social network analysis [11], [13].

5. Quality Functions

Once you've detected communities in a network, you need some way to tell if they're actually good. This is where quality functions come in. They give us quantitative measures to evaluate how well a partition captures the underlying structure. These measures fall into two broad categories: those that require known ground-truth labels (external validation) and those that don't (internal validation). In practice, most studies use both types to get a fuller picture of algorithm performance.

a) Modularity

Modularity is probably the most well-known quality function in community detection. Newman and Girvan introduced it in 2004 [9], and it's been the foundation for many popular algorithms since then including Louvain and Leiden.

The idea behind modularity is pretty intuitive. You compare the number of edges you see inside communities against what you'd expect if the edges were randomly placed, keeping the same degree distribution. If the actual number is significantly higher, that suggests real community structure.

The formula looks like this:

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j)$$

- A_{ij} is 1 if there's an edge between nodes i and j , 0 otherwise
- k_i and k_j are the degrees of nodes i and j
- m is the total number of edges
- $\delta(c_i, c_j)$ is 1 if i and j are in the same community, 0 otherwise
- The term $\frac{k_i k_j}{2m}$ is the expected number of edges between i and j in a random graph

Modularity values range from -1 to 1. Values close to 1 mean strong community structure lots of internal edges, few external ones. Values around 0 suggest the network is basically random. Negative values are rare and usually mean the partition is worse than random.

The main reason modularity is so popular is that it doesn't require ground-truth labels, and it's computationally efficient. But it's not perfect. Fortunato [4] pointed out a major limitation: modularity struggles to detect small communities in large networks. This is called the "resolution limit." Basically, when you optimize modularity, the algorithm tends to merge smaller communities into larger ones because it increases the overall score even if the smaller communities are actually meaningful.

Several researchers have proposed solutions to this problem, including multi-resolution modularity and stability-based measures [6], [10], [14]. These let you adjust a resolution parameter to detect communities at different scales.

b) Normalized Mutual Information (NMI)

When you have ground-truth labels like with benchmark datasets such as Karate Club or Football you need a way to measure how similar your detected communities are to the true ones. That's where NMI comes in.

NMI comes from information theory. It measures how much information the detected partition shares with the ground truth. The formula is:

$$NMI = \frac{-2 \sum_{i=1}^C \sum_{j=1}^{C'} N_{ij} \log \left(\frac{N_{ij} N}{N_i N_j} \right)}{\sum_{i=1}^C N_i \log \left(\frac{N_i}{N} \right) + \sum_{j=1}^{C'} N_j \log \left(\frac{N_j}{N} \right)}$$

- N is the total number of nodes
- C is the number of communities in the detected partition
- C' is the number of communities in the ground truth
- N_{ij} is the number of nodes in community i (detected) and community j (ground truth)
- N_i and N_j are the sizes of communities i and j

NMI ranges from 0 to 1. A score of 1 means perfect agreement your detected communities match the ground truth exactly. A score of 0 means the two partitions are completely independent. What makes NMI useful is that it doesn't favor partitions with more communities. Some metrics are biased toward over-segmentation, but NMI avoids that problem. Yang et al. [11] used NMI in their comparative study and found it to be one of the most reliable metrics for evaluating community detection algorithms.

c) Adjusted Rand Index (ARI)

ARI is another external validation metric, but it works differently from NMI. Instead of information theory, it's based on counting pairs of nodes.

The Rand Index measures the proportion of node pairs that are classified consistently in both partitions meaning they're either together in both clusterings or separated in both. The Adjusted Rand Index then corrects this for chance, comparing the Rand Index to what you'd expect from random clustering.

$$ARI = \frac{RI - \mathbb{E}[RI]}{\text{Max } RI - \mathbb{E}[RI]}$$

- RI is the Rand Index
- $\mathbb{E}[RI]$ is the expected Rand Index under random clustering
- Max RI is the maximum possible Rand Index

ARI ranges from -1 to 1. A value of 1 means perfect agreement, 0 means agreement expected by chance, and negative values mean agreement worse than chance.

The advantage of ARI over other metrics is that it accounts for chance agreement. This makes it more reliable than simple accuracy measures, especially when the number of communities differs between the two partitions. Gates et al. [27] recommend using both NMI and ARI together, since they capture different aspects of clustering quality.

V. Classical Community Detection Algorithms

Classical community detection algorithms form the foundation of modern network analysis and have been extensively studied and applied across various domains [11]. These methods can be categorized based on their underlying principles: modularity optimization, random walk-based approaches, label propagation, spectral methods, and flow simulation. This section provides a comprehensive overview of the most prominent classical algorithms and their characteristics.

1. Modularity-Based Algorithms

a) Louvain Algorithm

The Louvain method, introduced by Blondel et al., is one of the most widely adopted algorithms for community detection in large networks [24]. It operates through a greedy optimization approach that maximizes modularity, a quality function measuring the density of edges within communities compared to edges between communities.

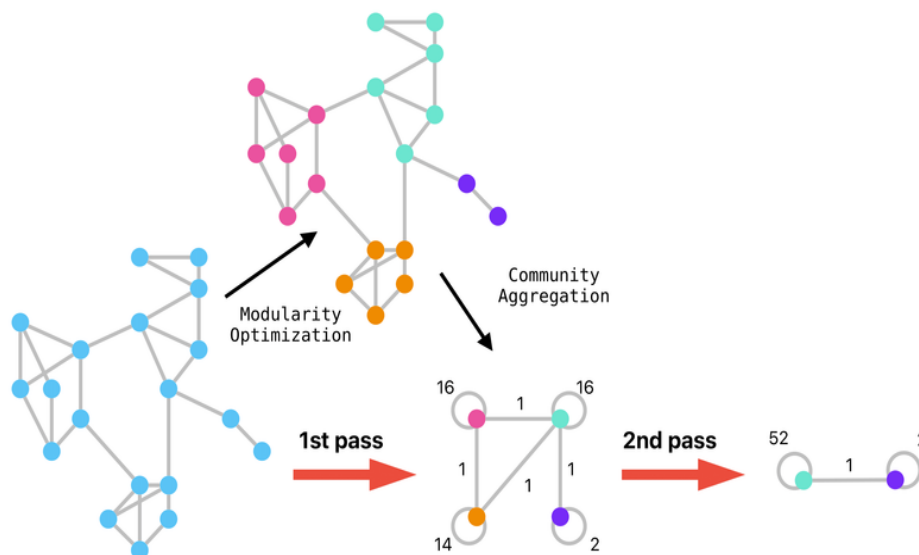


Figure 11 : Louvain Algorithm

The algorithm runs in two repeating stages. In the first phase, each node is initially assigned to its own community. The algorithm then evaluates the modularity gain obtained by moving each node to neighboring communities, selecting the move that yields the highest positive gain. This process continues until no further improvement is possible. In the second phase, a new network is constructed where nodes represent the communities identified in the first phase, and edges between these super-nodes are weighted by the sum of edges between nodes in the corresponding communities [11].

The Louvain algorithm's primary strengths lie in its computational efficiency and scalability, making it suitable for networks with millions of nodes. It runs in near-linear time complexity and consistently produces high modularity values [26]. However, the algorithm suffers from significant limitations. Research has demonstrated that Louvain may yield arbitrarily badly connected communities, with up to 25% of communities being poorly connected and up to 16% being internally disconnected, especially when the algorithm is applied iteratively [24]. This occurs when nodes acting as bridges between groups are moved to new communities, leaving the original community fragmented into disjoint subgraphs.

b) Leiden Algorithm

The Leiden algorithm was specifically developed to address the fundamental deficiencies of the Louvain method. Introduced by Traag et al. [24] in 2019, it guarantees that all detected communities are well-connected and provides explicit mathematical guarantees on partition quality.

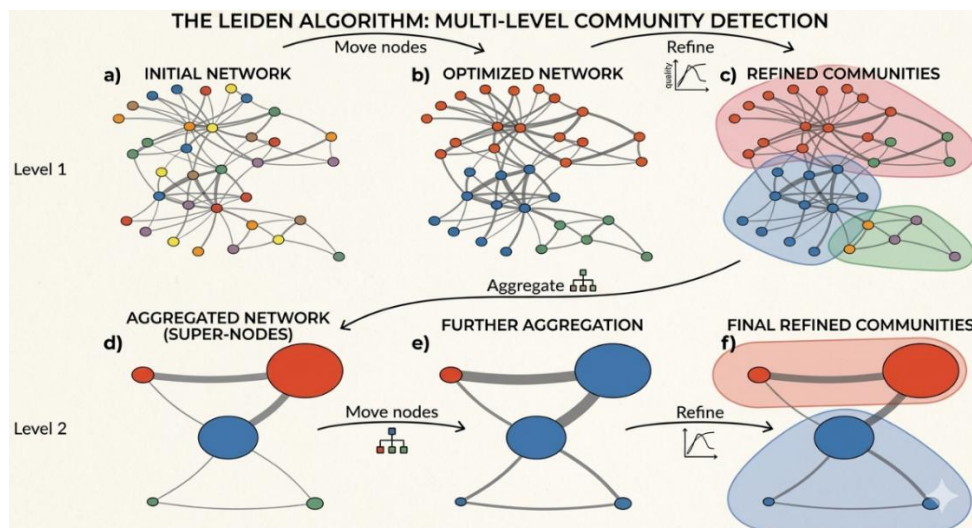


Figure 12 : Leiden Algorithm

The Leiden algorithm incorporates three key innovations. First, it introduces a refinement phase that subdivides communities and then moves nodes between these refined partitions, ensuring internal connectivity. Second, it uses a smart local move strategy that considers only nodes whose neighborhoods have changed, reducing unnecessary computations. Third, when constructing the aggregate network for the next iteration, Leiden merges only nodes within well-connected subsets of communities rather than entire communities [24].

Comparative studies show that Leiden consistently outperforms Louvain across multiple metrics [25]. It achieves higher modularity values (typically 0.7453 versus 0.5942 for Louvain on real-world networks), demonstrates significantly shorter execution times (averaging 22.52 seconds compared to

Louvain's longer runtimes on Twitter networks), and provides more stable partitions with better-defined community boundaries [25]. The algorithm converges to a partition where all subsets of all communities are locally optimally assigned, a property that Louvain cannot guarantee [24]. In a case study analyzing Twitter communities during the 2022 war in Ukraine, Sliwa et al. [25] found that Leiden detected substantially more communities (97 compared to Louvain's 41) and produced partitions with higher average modularity scores.

2. Random Walk-Based Algorithms

a) Walktrap Algorithm

The Walktrap algorithm, proposed by Pons and Latapy, leverages the intuition that random walks on a graph tend to remain trapped within densely connected regions [27]. The algorithm defines distances between vertices based on random walk transition probabilities and uses hierarchical agglomerative clustering to merge communities.

The method begins by computing transition probabilities for random walkers of a specified length (typically 4 steps). The distance between two nodes is calculated based on the similarity of their random walk probability distributions to all other nodes in the network. Communities are then identified by iteratively merging the pair of communities that results in the smallest increase in within-community distances [11].

Walktrap demonstrates strong performance on networks with clear flow-based community structure [27]. Studies indicate it achieves relatively better accuracy and smaller standard deviations for larger networks ($N > 1000$) and maintains stable performance across varying network densities [11], [26]. The algorithm has time complexity of $O(n^2m)$ or $O(n^2 \log n)$ on sparse networks. It is particularly well-suited for applications involving transmission of information, communication, behaviors, or infectious disease spread, as it naturally captures flow dynamics [27]. However, Walktrap's hierarchical nature means it requires selection of an appropriate cut point in the dendrogram to obtain the final partition.

b) Infomap Algorithm

Infomap, developed by Rosvall and Bergstrom, approaches community detection from an information-theoretic perspective. The algorithm seeks to minimize the map equation, which represents the expected description length of a random walk on the network using a two-level coding scheme [27].

Essentially, if a network has community structure, a random walker will spend long periods within communities before occasionally jumping between them. Infomap exploits this by assigning short code words to nodes within the same community (using community-specific codebooks) and longer codes for inter-community transitions. The optimal partition is the one that minimizes the total information needed to describe the random walker's trajectory [11].

Infomap consistently ranks among the top-performing algorithms in comparative studies [11], [26], [27]. For small networks ($N \leq 1000$), it achieves high accuracy with very small standard deviations, demonstrating excellent repeatability. The algorithm has linear time complexity $O(m)$ and scales well to large networks [26]. It shows a steep change in performance at a mixing parameter around $\mu = 0.6$, after which accuracy degrades [11]. Infomap is particularly effective for directed networks and networks with well-defined flow patterns. Studies comparing it to Walktrap show high overlap in detected communities (Adjusted Rand Index of 0.72), though Infomap typically requires fewer iterations to converge and does not require manual selection of partition granularity [26].

c) Markov Clustering (MCL)

The Markov Clustering algorithm simulates flow in a graph using two operators: expansion and inflation. The expansion operator computes random walks of higher length by raising the adjacency matrix to a power, allowing nodes to reach distant neighbors. The inflation operator strengthens strong connections and weakens weak ones by raising matrix elements to a power and renormalizing columns [28].

MCL iteratively applies expansion and inflation until the matrix converges to a form interpretable as disconnected components, which represent communities. Unlike many algorithms, MCL does not require pre-specification of the number of communities the partition emerges naturally from network topology [28].

However, MCL's performance heavily depends on the inflation parameter setting, making it sensitive to parameter selection. Studies show that while MCL performs well on synthetic networks and biological networks where it was originally developed, it generally produces results inferior to Louvain and Infomap on social networks [26], [28]. Recent work by Papadopoulos et al. [28] has demonstrated that pre-weighting the network using geometric or similarity measures significantly improves MCL's community detection performance. Their experiments show that geometrically inspired pre-weighting can increase the Normalized Mutual Information (NMI) of MCL from approximately 0.4 to 0.8 on certain benchmark networks, bringing it closer to current state-of-the-art methods [28].

3. Statistical Physics-based Methods

a) SpinGlass Algorithm

The SpinGlass algorithm, inspired by statistical physics, formulates community detection as finding the ground state of a Potts spin glass model. Each node is assigned to a spin value, and the algorithm minimizes an energy function that favors configurations where connected nodes have the same spin.

The method uses simulated annealing to explore the configuration space and find low-energy states corresponding to good community partitions. For small networks ($N \leq 1000$), SpinGlass achieves high accuracy comparable to Infomap and Walktrap [11], [26]. However, its computational complexity is prohibitively high for large networks, with running times that make it impractical for networks beyond a few thousand nodes. The algorithm is typically limited to networks smaller than 10,000 nodes due to computational constraints [26].

4. Divisive Algorithms

a) Girvan–Newman (GN) Algorithm

The Girvan-Newman algorithm is one of the earliest and most influential divisive methods for community detection [11]. It operates by iteratively identifying and removing edges with high betweenness centrality edges that lie on many shortest paths between node pairs.

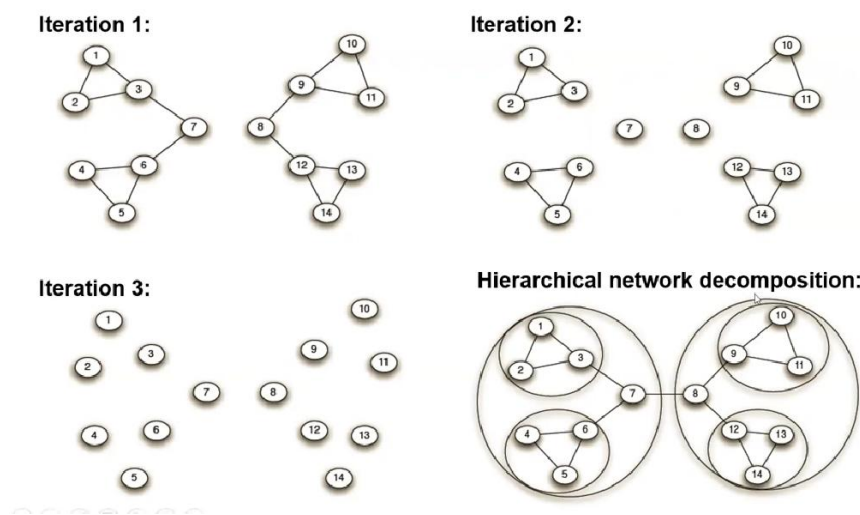


Figure 13 : Girvan Newman Algorithm

The algorithm begins with the complete network and repeatedly calculates edge betweenness for all edges, removes the edge with highest betweenness, and recalculates betweenness values. This process continues until all edges are removed or a desired number of communities is reached. A dendrogram captures the hierarchical community structure, and the partition with maximum modularity is typically selected [11].

Despite its historical importance and conceptual elegance, the GN algorithm has severe computational limitations. Its time complexity is $O(m^2n)$ or $O(n^3)$ on sparse graphs, making it impractical for networks larger than a few thousand nodes [26]. Studies show it cannot complete execution on networks with 20,000+ nodes even after multiple days of computation [11]. Additionally, the algorithm's performance degrades when multiple edges have identical betweenness values, creating ambiguity in edge selection. For these reasons, GN has largely been superseded by more efficient methods, though it remains valuable for small networks and as a conceptual baseline [27].

b) Kernighan-Lin Algorithm

The Kernighan-Lin algorithm is one of the earliest heuristic methods for graph partitioning, introduced by Kernighan and Lin in 1970 [43]. Unlike modern modularity optimization methods, this algorithm was designed to partition a graph into two equal-sized subsets while minimizing the number of edges crossing between them.

The algorithm works through an iterative greedy strategy. It begins with an arbitrary initial partition of the nodes into two groups of specified sizes. At each step, the algorithm evaluates all possible swaps of node pairs between the two groups, calculating the gain in cut size for each swap. The pair that produces the largest reduction is swapped, even if the gain is negative, allowing the algorithm to escape local optima. This process continues until no further improvement is possible [43].

Kernighan-Lin algorithm finds high-quality partitions through its sophisticated local search strategy. However, it has several limitations. The algorithm requires the number of communities to be specified in advance and is typically designed for bisection rather than arbitrary community numbers. It also has relatively high computational complexity, making it impractical for very large networks. Despite these limitations, the Kernighan-Lin algorithm remains influential as one of the foundational methods in graph partitioning and community detection [43].

Several extensions have been proposed to address its limitations. The Combo algorithm combines merging, splitting, and node-moving strategies to optimize modularity across different objective functions. The Modified Fine-Tuning Strategy adds an additional refinement step to the recursive bisection process, allowing nodes to move across existing communities after the standard bisection, which improves accuracy without increasing computational complexity [43].

5. Label Propagation Methods

a) Label Propagation Algorithm (LPA)

The Label Propagation Algorithm is a near-linear time method based on local information diffusion [27]. Each node is initially assigned a unique label. In each iteration, nodes adopt the label that is most frequent among their neighbors, with ties broken randomly or using node degree as a tiebreaker.

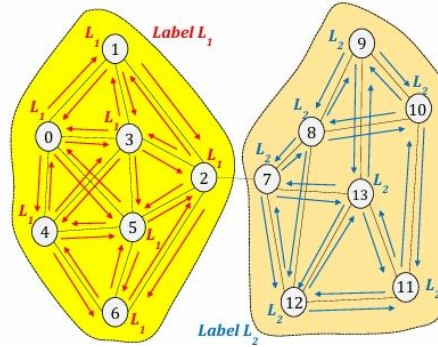


Figure 14 : LPA Algorithm

The process continues until labels stabilize, meaning no node changes its label. Nodes sharing the same final label belong to the same community. LPA's primary advantage is its simplicity and speed it runs in $O(m)$ time and requires no parameter tuning or prior knowledge of community number [11], [26].

However, LPA suffers from significant stability issues. Different runs on the same network can produce dramatically different results due to its stochastic nature [11], [26]. Studies report that LPA has much larger standard deviations than other algorithms, with outputs that are fundamentally unstable. The algorithm frequently converges to a single giant community rather than meaningful partitions, particularly in networks lacking strong community structure [26]. While some nodes quickly reach consensus, others may oscillate between labels indefinitely. Gates et al. [27] note that LPA's lack of stability makes it problematic for applications requiring reproducible results. Despite these limitations, LPA achieves competitive performance on certain benchmark networks, particularly when combined with memory mechanisms or other enhancements [11].

b) Fast Label Propagation Algorithm

FLPA introduces optimizations to the standard LPA to improve convergence speed and stability. It employs techniques such as asynchronous updates (where nodes update labels immediately rather than synchronously), preferential treatment of high-degree nodes in the update order, and strategic initialization of labels based on network topology [26].

Some variants incorporate a memory mechanism (MemLPA) where nodes remember past label states and use this historical information in decision-making. Experiments show that MemLPA outperforms

standard LPA for mixing parameters $\mu \in [0.5, 0.7]$ and performs comparably to Infomap and Walktrap in this range [11]. For $\mu \geq 0.7$, MemLPA demonstrates superior performance compared to most algorithms except edge betweenness methods.

However, even with these improvements, FLPA variants still exhibit some instability, though less severe than standard LPA [26]. The memory-based approaches tend to find a greater number of smaller communities, which can affect partition quality metrics depending on the application requirements.

c) Label Propagation Algorithm with Modularity and Node Importance (LPA-MNI)

The standard Label Propagation Algorithm suffers from randomness and instability, as different runs can produce dramatically different community partitions. LPA-MNI addresses these issues by integrating modularity optimization and node importance into the label propagation framework [43].

The algorithm operates in two main stages. First, it identifies initial communities through modularity maximization, assigning unique labels to nodes within these initial communities. This initialization step significantly reduces the method's susceptibility to randomness. Second, node importance, determined by degree centrality, dictates the order of label updates. Nodes with higher importance are updated first, ensuring a systematic and stable propagation process [43].

Extensive testing on real-world and synthetic networks demonstrates that LPA-MNI achieves enhanced accuracy, robustness, and efficiency compared to the standard LPA. The method produces higher modularity values and more consistent community numbers, confirming its effectiveness. The introduction of a deterministic and accurate label update mechanism significantly improves upon the traditional label propagation method's stability, making it a more reliable choice for practical applications [43].

d) Constrained Label Propagation Algorithm (CLPA)

CLPA reformulates the label propagation algorithm as an optimization problem to address its inherent drawbacks. The key innovation is the introduction of a modified objective function that incorporates constraints to penalize less desirable solutions, such as the trivial solution where all vertices share the same label [43].

The algorithm proposes several constraint variants tailored to different network types. A modularity-maximizing variant focuses on optimizing community quality, while specialized adaptations exist for bipartite networks. Through comparative analysis of both synthetic and real-world networks, CLPA demonstrates its ability to evade the pitfalls of standard LPA while showing versatility in handling different network structures [43].

This methodology enables the detection of more meaningful communities by effectively leveraging constraints within the optimization process. CLPA offers a nuanced and flexible tool for network analysis, providing researchers with greater control over the community detection process through its constraint-based formulation [43].

e) FluidC Algorithm

FluidC (Fluid Communities) is a propagation-based algorithm that treats communities as fluids expanding through the network. Each community is initialized at a random seed node and grows by incorporating neighboring nodes based on density and flow dynamics [26].

The algorithm simulates the process of fluids flowing and displacing each other, where denser regions (communities) naturally emerge. Like LPA, FluidC is computationally efficient with complexity near-linear time. It performs particularly well on large-scale networks; studies show it achieves high and stable performance (ARI and Purity around 0.9) on LFR benchmark networks with up to 500,000 nodes [26].

However, FluidC requires the number of communities as an input parameter, which is often unknown in real-world scenarios [26], [27]. Additionally, the algorithm produces different results across runs due to its stochastic initialization and propagation process, though this instability is less pronounced than in standard LPA. On some networks, FluidC's ARI values drop nearly to zero, indicating highly inconsistent partitions depending on initialization [26].

6. Spectral Methods

a) Eigenvector-Based Method

Spectral methods leverage the eigenvectors and eigenvalues of matrices derived from the graph, such as the adjacency matrix, Laplacian matrix, or modularity matrix [11]. Newman's leading eigenvector method is a prominent example that uses the eigenvector corresponding to the largest positive eigenvalue of the modularity matrix to recursively bisect the network.

The algorithm computes the modularity matrix B , where $B_{\{ij\}} = A_{\{ij\}} - \frac{(k_i k_j)}{2m}$ with A being the adjacency matrix, k representing node degrees, and m the total number of edges. The leading eigenvector is calculated, and nodes are assigned to communities based on the sign of their corresponding eigenvector components. This process is applied recursively to each subcommunity until modularity cannot be further improved [11].

Spectral methods have theoretical guarantees under certain network models (such as the stochastic block model) and can detect communities with clear structural separation [27]. However, comparative studies show that the leading eigenvector method tends to overshoot results and perform worse than optimization-based methods like Louvain [11], [26]. The method has time complexity of $O(n(m+n))$ or $O(n^2)$ on sparse networks.

Recent advances have combined spectral methods with other techniques. For instance, using node embeddings (like Node2Vec) followed by spectral clustering shows improved performance, particularly when enhanced with neural network-based embeddings for networks with high clustering coefficients [26]. Spectral approaches also face challenges in determining the appropriate number of communities and can struggle with networks containing overlapping or hierarchical community structures [27].

7. Seed-centric Approaches

a) SimCMR (Similarity-Based Community Merge/Refinement)

SimCMR represents a class of algorithms that combine multiple strategies for community detection [26]. These methods typically begin with an initial partition (often from a fast algorithm) and then refine it by merging or splitting communities based on similarity measures.

The approach involves computing node or community similarities using various metrics (structural equivalence, random walk probabilities, or embedding distances) and iteratively improving the partition by merging similar communities or refining boundary nodes. Some variants use a two-stage process: first detecting candidate communities quickly, then refining boundaries using more sophisticated similarity computations [26].

SimCMR-type approaches can incorporate both global and local information, balancing accuracy with computational efficiency. They often achieve better stability than pure propagation methods while maintaining reasonable runtime. However, the performance depends heavily on the choice of similarity metric and the initial partition quality [26].

VI. Modern Approaches

While classical algorithms have been the foundation of community detection for many years, modern approaches have emerged to overcome their limitations in terms of scalability, expressiveness, and the ability to leverage node attributes. These methods fall into two major categories: graph embedding techniques, which learn low-dimensional vector representations of nodes, and deep learning approaches, which use neural network architectures specifically designed for graph-structured data.

1. Graph Embeddings

Graph embedding methods aim to represent each node of a network as a dense, low-dimensional vector in a continuous space such that structurally similar nodes are mapped close together. These representations can then be used with standard machine learning techniques, such as k-means clustering, to perform community detection [7], [8].

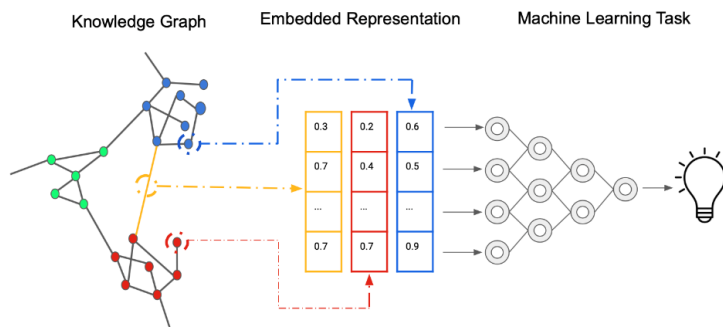


Figure 15 : Illustration of the graph embedding pipeline from knowledge graph to feature vectors.

a) DeepWalk

DeepWalk was one of the first methods to apply natural language processing techniques to graph representation learning. Proposed by Perozzi et al. in 2014, it draws an analogy between sentences in a corpus and sequences of nodes visited during random walks on a graph [29].

The algorithm works in two steps. First, it generates many truncated random walks starting from each node in the graph. These walks produce sequences of nodes that reflect the local neighborhood structure. Second, these sequences are fed into the Word2Vec Skip-gram model, which learns node embeddings by predicting the context nodes surrounding each node within a fixed window [29].

The idea behind DeepWalk is that nodes which frequently co-occur in random walks tend to be structurally related, often belonging to the same community. As a result, their embeddings end up close together in the vector space. Once embeddings are learned, standard clustering algorithms such as k-means can be applied to group nodes into communities [29].

DeepWalk scales well to large networks and runs efficiently in an online manner, making it suitable for dynamic graphs. However, its random walks are entirely uniform, meaning it does not distinguish between exploration of different structural properties. Additionally, DeepWalk is a transductive method, which means it cannot directly generate embeddings for unseen nodes without retraining [8].

b) Node2Vec

Node2Vec, introduced by Grover and Leskovec, extends DeepWalk by introducing a more flexible and controllable random walk strategy. The main contribution is a biased second-order random walk mechanism governed by two parameters: the return parameter p and the in-out parameter q [29].

The parameter p controls the likelihood of immediately revisiting a node already visited, effectively regulating how much the walk explores or returns to its starting area. The parameter q balances the walk between two different strategies: Breadth-First Sampling (BFS), which explores the immediate neighborhood and captures homophily, and Depth-First Sampling (DFS), which moves further away from the starting node and captures structural equivalence. By tuning these two parameters, Node2Vec can produce embeddings that reflect either community membership (when p is high and q is low) or structural roles within the network (when p is low and q is high) [29].

Comparative studies show that Node2Vec consistently outperforms DeepWalk on community detection tasks, particularly on networks where both community membership and structural roles are important. However, like DeepWalk, Node2Vec is transductive and does not generalize to unseen nodes. Its performance also depends on careful tuning of p and q , which may require domain knowledge or cross-validation [8].

c) GraphSAGE

GraphSAGE (Graph Sample and aggreGatE) addresses a key limitation of DeepWalk and Node2Vec by introducing an inductive learning framework. Proposed by Hamilton et al., it learns an aggregation function over node neighborhoods rather than learning fixed embeddings for each node [30].

At inference time, the embedding of a node is computed by sampling a fixed number of neighbors and applying a learned aggregation function (such as mean pooling, max pooling, or an LSTM) to combine the features of those neighbors. This process is applied over multiple hops, allowing information from further parts of the graph to be incorporated. Because the aggregation function is shared across all nodes, GraphSAGE can generate embeddings for nodes that were not present during training, making it inherently inductive [30].

For community detection, the embeddings produced by GraphSAGE can be clustered using standard algorithms. Its ability to incorporate both structural information and node features (such as user profile attributes) makes it especially useful in attributed social networks. Hamilton et al. demonstrated that GraphSAGE significantly outperforms DeepWalk and Node2Vec on node classification and link prediction tasks in large-scale graphs such as Reddit and Pinterest [30].

2. Deep Learning on Graphs

Deep learning approaches for graphs extend classical neural network architectures to handle the irregular, non-Euclidean structure of graph data. Unlike images or text sequences, graphs have no fixed spatial locality or ordering, which makes conventional convolutional or recurrent networks unsuitable. A family of models known as Graph Neural Networks (GNNs) addresses this challenge by defining learnable operations that propagate and aggregate information along graph edges [15].

a) Graph Neural Networks (GNN)

Graph Neural Networks (GNNs) constitute a broad class of models that learn representations for nodes, edges, or entire graphs by iteratively aggregating information from local neighborhoods. The core idea, shared across most GNN variants, is a message-passing mechanism: at each layer, every node collects feature vectors from its neighbors, aggregates them (typically through summation or averaging), and updates its own representation [15].

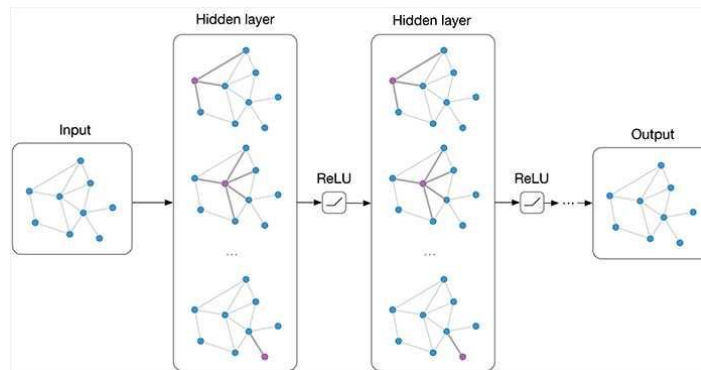


Figure 16 : Architecture of a Multi-layer Graph Neural Network.

After k layers, the representation of a node encodes information from its k -hop neighborhood. This multi-hop aggregation makes GNNs naturally suited for community detection, as nodes belonging to the same community tend to have similar neighborhood structures. GNNs can be trained in a supervised manner using labeled node data, in a semi-supervised manner with partial labels, or in an unsupervised manner using reconstruction-based objectives [15].

b) Graph Convolutional Networks (GCN)

Graph Convolutional Networks (GCNs), introduced by Thomas Kipf and Max Welling, are among the most influential and widely used variants of Graph Neural Networks (GNNs). They extend the concept of convolution from regular grid-structured data, such as images, to irregular graph structures by relying on principles from spectral graph theory [30].

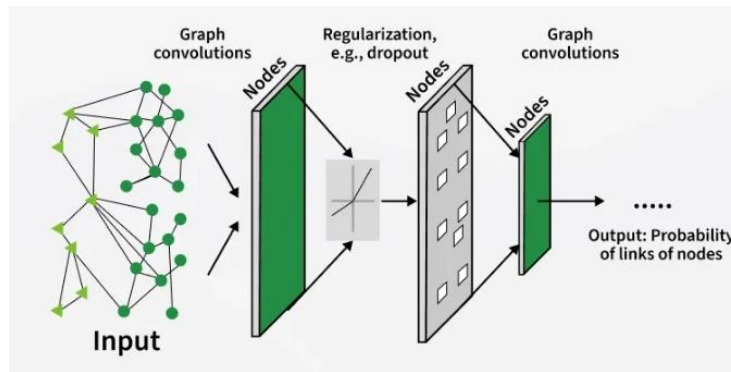


Figure 17 : Schematic representation of GCN layers.

In a GCN layer, each node updates its representation by combining its own features with those of its neighboring nodes. This aggregation is performed in a normalized manner to ensure stability and balanced contributions from different nodes, followed by a non-linear transformation. By stacking multiple layers, the model can capture information from progressively larger neighborhoods, enabling it to learn both local and global structural patterns in the graph [30].

For community detection tasks, GCNs are often employed within an encoder–decoder framework. The encoder uses graph convolution operations to transform nodes into a latent embedding space, while the decoder leverages these embeddings to reconstruct the graph structure or to infer community memberships. This approach has shown strong performance on benchmark datasets, often producing more coherent and semantically meaningful communities compared to traditional methods, especially when node features are available [15], [30].

Despite their effectiveness, GCNs present several challenges in practice. One major issue is over-smoothing, where node representations become increasingly similar as more layers are added, reducing their discriminative power. Additionally, scaling GCNs to large graphs can be computationally demanding. To address these limitations, recent research has introduced techniques such as neighbor sampling, mini-batch training, and graph partitioning, which improve both efficiency and scalability [15].

c) Graph Attention Networks (GAT)

Standard Graph Convolutional Networks treat all neighbors equally when updating a node's representation. Graph Attention Networks improve upon this by learning to assign different levels of importance to different neighbors, as surveyed by Su et al. [42].

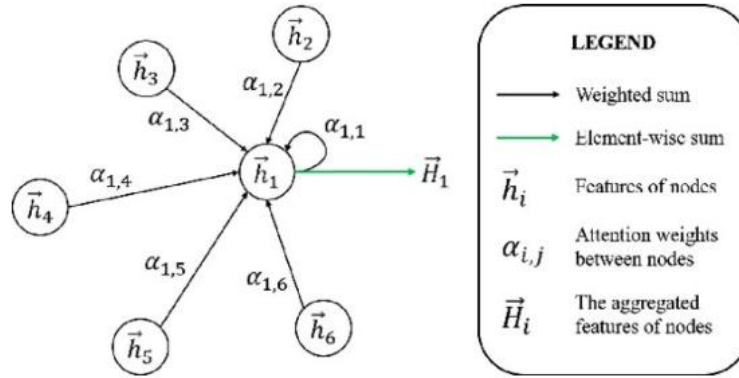


Figure 18 : Graph Attention Network

The key idea is that not all connections in a social network matter the same way. A close friend should have more influence on a user's community membership than a casual acquaintance. GAT automatically learns these importance weights through an attention mechanism. For each node, the model calculates how much attention to pay to each neighbor, then aggregates neighbor information accordingly.

For community detection, GAT offers several practical advantages. First, it can filter out noisy or irrelevant connections that might otherwise mislead the detection process. Second, it handles networks with multiple relationship types, such as friendship, family, and professional connections on the same platform. Third, the learned attention weights can be interpreted, revealing which types of relationships are most predictive of community belonging [42].

Several researchers have successfully applied GAT to community detection. DMGI uses attention to combine information from different types of relationships in multiplex networks. MAGNN applies attention along metapaths in heterogeneous networks, capturing complex semantic relationships. HDMI extends this approach with semantic attention to fuse embeddings across multiple network layers [42].

d) Autoencoder-Based Methods

Autoencoders are a popular deep learning architecture for unsupervised community detection. They learn compact node representations by compressing and reconstructing a network's adjacency matrix through an encoder-decoder structure with a narrow bottleneck. Nodes that share similar connectivity patterns end up with similar compressed representations, enabling standard clustering algorithms like k means to effectively detect communities [42].

Several important variants exist. Stacked autoencoders use multiple hidden layers to learn hierarchical features. Denoising autoencoders are trained on corrupted versions of the input, making them robust to missing or noisy edges. Variational autoencoders learn probability distributions over embeddings rather than fixed points, capturing uncertainty in community assignments [42].

Specific methods include Semi DRN, which incorporates modularity optimization into autoencoder training, DANE, which uses two parallel autoencoders for network structure and node attributes, and MGAE, which marginalizes over multiple corrupted inputs for greater robustness. Autoencoders are particularly well suited for social network community detection because they require no labeled training data, scale efficiently to large networks, and can easily incorporate multiple information sources such as user profiles and interaction histories alongside network structure [42].

e) Variational Graph Autoencoders

While the previous section discussed deterministic autoencoders for community detection, the Variational Graph Autoencoder (VGAE), introduced by Kipf and Welling, extends this framework with a probabilistic approach [4]. Unlike standard autoencoders that learn fixed embeddings, VGAE learns a probability distribution over the latent space, capturing uncertainty in node representations and community assignments [1].

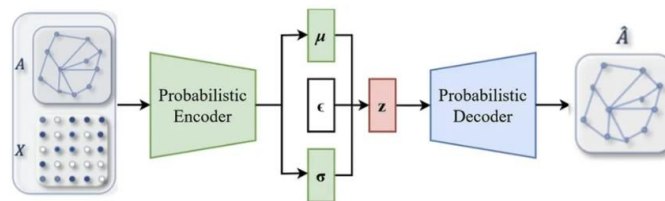


Figure 19 : Architecture of a VGAE with encoder and decoder

The architecture follows the same encoder-decoder principle as deterministic autoencoders, but with a key difference: the encoder outputs parameters of a Gaussian distribution (mean and variance vectors) rather than fixed embeddings. Training optimizes a variational lower bound that balances reconstruction accuracy against a KL divergence regularizer, which prevents overfitting and promotes smooth latent spaces [4].

For community detection, VGAE offers advantages over deterministic autoencoders. The probabilistic representation naturally handles ambiguous community boundaries, where nodes at the periphery of communities exhibit higher variance. Choong et al. demonstrated that VGAE with dual optimization achieves competitive community detection performance with faster convergence compared to standard autoencoders [1]. Murata's research further validated that encoding graph structure using multiple Gaussian distributions enables effective unsupervised community extraction [3].

3. Advantages and Limitations of Community Detection Algorithms

People often don't think enough about which algorithm to pick for community detection. They just grab whatever's popular or easiest to implement. But the choice really matters because different algorithms behave differently depending on the network. Some are fast but unstable, others are accurate but slow. Some work well on small graphs but fall apart at scale. In this section, we will go through the main approaches and discuss what they're actually good for and where they cause problems.

a) Modularity-Based Algorithms

Louvain and Leiden are everywhere these days. Almost everyone uses them. They're fast, they scale, and they give decent results most of the time. You also don't need to specify the number of communities upfront, which is useful because you usually don't know that in advance.

The downside is the resolution limit. These algorithms struggle to find small communities in large networks [4]. They also tend to produce balanced partitions even when the real structure is unbalanced. Louvain has another issue it can produce disconnected communities on large graphs [24]. Leiden fixes this, but it's still sensitive to the initial random seed.

b) Random Walk-Based Algorithms

Infomap looks at community detection differently it uses information theory. It works well on directed networks and networks where flow matters [11], [27]. Walktrap is decent for medium-sized graphs. MCL was designed for biological networks and still does well there.

The problems? Walktrap needs you to decide where to cut the dendrogram, which adds another decision. Infomap can be slow on very large graphs. MCL is sensitive to its inflation parameter, and if you don't tune it, performance drops [28]. In most comparisons, these methods don't beat Louvain and Leiden [26].

c) Label Propagation Algorithms

Speed is the main reason people use label propagation. LPA runs in near-linear time and doesn't need parameters. FluidC is also very fast and works on large graphs.

But stability is a real issue. LPA gives different results almost every time you run it [11], [26]. It often collapses into one giant community when the structure isn't clear. FluidC needs the number of communities as input, which you usually don't have. Enhanced versions like LPA-MNI and CLPA are more stable but slower [43].

d) Divisive Algorithms

Girvan-Newman is elegant and historically important. It gives you a hierarchical view and doesn't need the number of communities upfront.

The computational cost kills it. The algorithm is $O(m^2n)$ on sparse graphs, so it's unusable for networks bigger than a few thousand nodes [26]. Studies show it can't finish on networks with 20,000 nodes even after days of computation [11]. Kernighan-Lin needs the number of communities and is mostly for splitting into two groups.

e) Spectral Methods

They're usually beaten by Louvain [11], [26]. Deciding how many communities to use is also tricky. They work better when combined with other techniques.

f) Graph Embedding Methods

These methods turn nodes into vectors, which you can later cluster using something like k-means. GraphSAGE is interesting because it's inductive it can generate embeddings for nodes it's never seen before without retraining [30]. Node2Vec gives you more control than DeepWalk because you can adjust the random walk behavior.

The thing is that these methods weren't really made for community detection. They're designed for node classification and link prediction. The embeddings capture structural similarity, but there's no guarantee that clustering them will give you meaningful communities. They also need careful tuning and can be expensive on large graphs.

g) Graph Neural Networks

GNNs are powerful because they can use both structure and node attributes. GAT's attention mechanism helps ignore noisy connections [42]. VGAE adds a probabilistic element that handles uncertainty in community assignments.

These methods need a lot of compute and careful training. Without node attributes, they often underperform compared to classical algorithms on large sparse networks [26]. Oversmoothing is a real problem as you add more layers, node representations become increasingly similar [15]. My own results in Chapter 4 confirm this: deep learning methods failed on large networks and performed poorly when community structure was ambiguous.

h) Autoencoder-Based Methods

Autoencoders are unsupervised and can handle multiple information sources. They're good for attributed networks [42].

Performance depends heavily on the architecture and training. They're compute-intensive and need careful tuning. Like other deep learning methods, they struggle to scale.

VII. Community Detection Applications

1. General Application Domains

People use community detection for all kinds of things. It's not just a theoretical concept. Researchers and practitioners apply these algorithms to problems ranging from biology to marketing to urban planning. In this section, we will go over some of the main areas where community detection has been useful and then explain why we chose recommender systems for this project.

a) Social Network Analysis

Social media platforms like Facebook, Twitter, and Instagram are basically giant graphs.

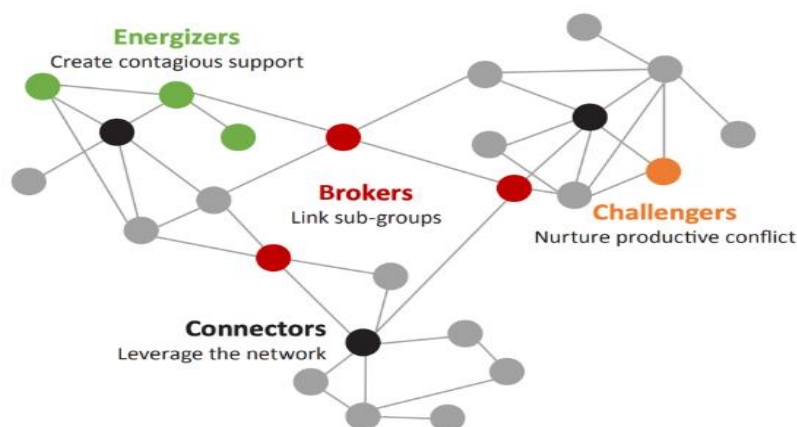


Figure 20 : Social Network Analysis

Users are nodes, and connections like follows or friendships are edges. Communities in these networks often map to real groups-friend circles, people with similar interests, or professional networks. Researchers have used community detection to find influential users (see Figure 20), study how information spreads, and understand how opinions form online [19]. Companies use these techniques too. They segment customers into groups and identify key opinion leaders who can help promote their products [17], [21].

b) Biological Networks

Community detection is also common in biology. Researchers apply it to protein-protein interaction networks, gene regulatory networks, and metabolic pathways [4].

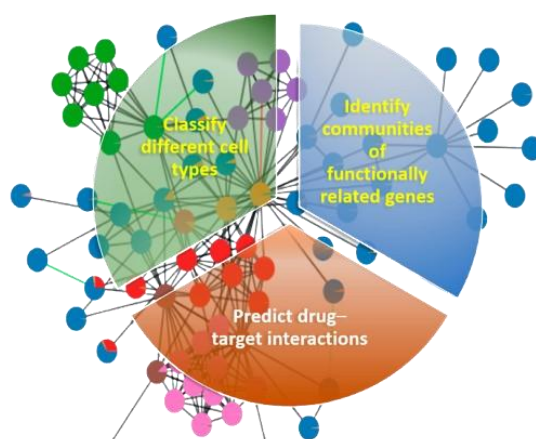


Figure 21 : Biological Network Analysis

In these systems, communities usually correspond to functional modules-groups of proteins or genes that work together to perform specific tasks in the cell (see Figure 21). MCL, which we mentioned earlier, is commonly used in bioinformatics for this kind of analysis [28]. Finding these modules helps biologists understand cellular processes and identify potential drug targets.

c) Infrastructure and Transportation Networks

Community detection shows up in physical networks too. Power grids, road networks, and communication systems can all be analyzed this way. In power grids, communities can reveal clusters of nodes that are tightly coupled and potentially vulnerable to cascading failures. In transportation networks, communities often correspond to geographic regions or clusters of stations with heavy traffic flow [4]. Urban planners use this information to optimize infrastructure and improve resilience.

2. Community Detection for Targeted Marketing and E-Commerce

Recent research demonstrates the practical value of community detection in marketing contexts. S. Paul *et al.* [17] introduced the Commerce Graph which is a dynamic network that links buyers, sellers, products, and transactions demonstrating that community detection using the Louvain algorithm enables effective customer segmentation. The study found that customers with higher centrality scores (degree, betweenness, closeness) play pivotal roles in the network, providing actionable insights for personalized promotions and optimized business operations.

In the domain of customer segmentation, Nasirzonouzi [21] proposed a bipartite graph model of customers and products using the "Online Retail II" dataset.

By converting the bipartite graph into a customer-to-customer network and applying community detection, the study revealed complex interrelations that traditional methods (K-means clustering) fail to capture effectively. Centrality metrics were used to quantify customer influence, enabling highly targeted marketing strategies based on network roles such as key influencers and bridging nodes.

A. Kanavos *et al.* [19] applied Louvain and Label Propagation algorithms to Twitter datasets to delineate community structures and identify influential users. Their findings demonstrate that community structures significantly influence information dissemination and user engagement, offering strategic insights for influencer engagement across marketing, politics, and public health sectors.

3. Integrating Textual Semantics

S. K. S. *et al.* [20] proposed a novel framework combining pre-trained language models (PLMs) and graph neural networks (GNNs) for community detection in text-attributed graphs. Using datasets from Goodreads (books, reviews) and Amazon (movies), the study demonstrates that integrating textual embeddings with graph structure significantly improves community detection accuracy for recommendation systems. This approach bridges the gap between semantic content and topological structure, enabling more nuanced community identification in text-rich domains.

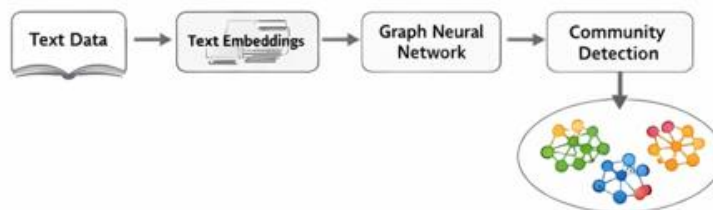


Figure 22 : Semantic-topological community detection framework

4. Community Detection for Recommender Systems

Community detection has emerged as a powerful approach for addressing fundamental challenges in recommender systems. Sánchez-Moreno et al. [18] addressed the cold-start problem (new users with no interaction history) and the gray-sheep problem (users with unusual preferences) in music recommender systems. Their proposed user similarity metric based on social network structure and user ratings captures homophily effects in implicit communities, significantly improving recommendation reliability for users with limited or atypical preference data.

Saikalyan et al. [22] proposed a recommender system using local community detection via personalized PageRank and the Bethe Hessian matrix to estimate community numbers, followed by non-negative matrix factorization for overlapping community detection. The approach defines a belongingness factor for each user-community relationship and effectively addresses the cold-start problem on MovieLens 1M and Facebook Ego Network datasets.

For GNN-based recommender systems, Hu et al. [23] introduced the General Graph Coarsening Recommendation Framework (GGCRF), which uses community detection to divide user-item graphs into smaller segments, forming super-nodes that encapsulate groups of similar nodes. This framework overcomes the limitation of GNNs that primarily focus on closely located nodes, addressing the long-tail effect prevalent in recommendation systems.

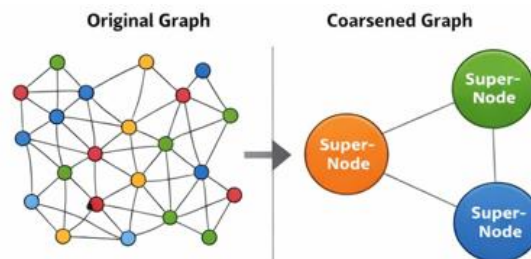


Figure 23 : Original vs Coarsened Graphs

In this project, community detection is applied to recommender systems for several reasons that align directly with my research objectives. First, the connection between community structure and user preferences is intuitive if users in the same community share similar tastes, then community membership should help generate better recommendations. This is especially valuable when individual user data is sparse, which is almost always the case in e-commerce. Second, communities offer a practical solution to the cold-start problem; new users with no history can be assigned to a community based on minimal information and receive recommendations from that community's collective preferences [18], [50].

Third, communities provide interpretability-recommendations can be explained as "other people in your community liked this," which builds user trust [49], [50]. Finally, as we will show in Chapter 4, combining communities with matrix factorization methods like SVD and ALS captures complementary aspects of user preferences, leading to significant improvements in recommendation performance.

VIII. Conclusion of Chapter

In this chapter, we covered the theoretical and algorithmic foundations of community detection. We started with graph theory fundamentals graph types, centrality measures, connectivity, and density which provide mathematical language for describing networks. We then defined what communities are and discussed the various quality functions, including modularity, NMI, and ARI, that researchers use to evaluate community detection results.

We reviewed the main classical algorithms, including modularity-based methods like Louvain and Leiden, random walk-based approaches like Walktrap and Infomap, label propagation methods, divisive algorithms, and spectral methods. We also covered modern approaches, including graph embeddings like DeepWalk and Node2Vec, and deep learning methods like GCNs, GATs, and autoencoders. For each approach, we discussed their strengths and limitations based on literature.

Finally, we surveyed the main application domains where community detection has been applied, including social network analysis, biological networks, infrastructure networks, information networks, marketing, and recommender systems.

The next chapter narrows the focus to community detection specifically for recommender systems. We cover the fundamentals of recommendation, review the existing work that combines community detection with recommendation, and present the hybrid approaches evaluated in our experiments.

Chapter 3: Community Detection for Recommender Systems

Chapter 3: Community Detection for Recommender Systems

I. Introduction to Chapter

In the previous chapter, we reviewed the state of the art in community detection the algorithms, their advantages and limitations, and their main application domains. Now we turn our attention to the specific focus of this research: using community detection to improve recommender systems.

This chapter begins with an overview of recommender systems, covering the main paradigms content-based filtering, collaborative filtering, and hybrid approaches-and the key challenges they face, such as data sparsity, the cold-start problem, and scalability. We then review the existing literature that has explored the intersection of community detection and recommender systems. Finally, we present the hybrid approaches we propose in this work, which combine community detection with matrix factorization methods like SVD, ALS, and BM25.

The goal of this chapter is to establish the theoretical and methodological foundation for the experimental work that follows in Chapter 4, where we implement and evaluate these approaches on real-world and synthetic datasets.

II. Foundations of Recommender Systems

1. Definition and Importance

A recommender system solves a specific version of the information overload problem given what a user has engaged with before, predict what they will want next. The practical stakes are high Netflix has reported that over 80% of content watched on the platform is driven by its recommendation engine, and Amazon attributes a significant share of revenue to its 'customers also bought' feature [35]. By personalizing content delivery, recommender systems improve user satisfaction, increase engagement, and drive business revenue. The core function of any recommender system is to take a user's known preferences and past behaviors as input, then generate a ranked list of items that the user is most likely to find appealing [36].

2. Classical Paradigms

Three main paradigms have historically dominated recommender system research and practice [35], [36].

Content-Based Filtering recommends items like those a user has liked in the past. This approach builds a profile for each User-Based on the features of items they have interacted with. For example, if a user frequently watches action movies starring a particular actor, the system will recommend other action movies featuring that same actor. Content-based filtering recommends new items that have no prior interactions. That said, overspecialization is a real problem, where recommendations become too narrow and fail to discover unexpected items that the user might still enjoy [35].

Collaborative Filtering is the most widely adopted paradigm. Instead of relying on item features, it leverages the "wisdom of the crowd": users who agreed in the past are likely to agree again in the future. Collaborative filtering comes in two main flavors. User-based collaborative filtering finds users with similar taste patterns to the target user and recommends items those similar users have liked. Item-based collaborative filtering finds items that are frequently liked together and recommends them to users who have shown interest in similar items. The primary strength of collaborative filtering is its ability to make recommendations in domains where item features are complex or unavailable.

Its main weakness is the cold-start problem: new users with no interaction history cannot be effectively matched to similar users, and new items cannot be recommended until someone interacts with them [36], [37].

Hybrid Approaches combine content-based and collaborative filtering to overcome their respective limitations. A hybrid system might use content-based profiles to address the cold-start problem for new items while relying on collaborative filtering for established items. Many commercial systems, including Netflix and Amazon, employ hybrid architectures to achieve robust performance across different scenarios [37].

3. Challenges in Recommender Systems

The gap between how recommender systems perform in controlled benchmarks and how they behave on real platforms comes down to four recurring problems [49] that no single method has fully solved:

- **Data sparsity:** In real-world platforms, the user–item interaction matrix typically contains less than 1% of possible entries, making it difficult to identify reliable similar users or items using traditional collaborative filtering [49], [50].
- **Cold start problem:** New users have no interaction history, and new items cannot be recommended until sufficient feedback accumulates. This issue is particularly severe in dynamic social network environments [50].
- **Scalability:** As user bases grow to millions, many recommendation algorithms become computationally prohibitive. The complexity of traditional collaborative filtering scales directly with the number of users and items [49], [51].
- **Interpretability:** Deep learning–based recommenders often operate as black boxes, reducing user trust. In contrast, community-based approaches can provide transparent explanations grounded in group membership [49].

Community detection does not solve these problems outright, but it offers a structural shortcut: instead of computing similarity across all user pairs, group membership narrows the candidate space and provides a fallback signal when interaction data is thin [50], [51].

4. Social Recommender Systems

Social recommender systems (SRS) extend traditional collaborative filtering by incorporating information extracted from social network services [50]. The underlying assumption is that users with social ties tend to exhibit similar preferences a phenomenon known as homophily [50].

Social signals used in SRS can be categorized as [50]:

- **Explicit ties:** Friendship, follow relationships, and trust statements explicitly declared by users on the social platform.
- **Implicit ties:** Behavioral similarities derived from co-rating patterns, shared content, or co-commenting activities, even when no direct social connection exists.

Gasparetti et al. [50] provide a comprehensive survey of social recommender systems, noting that implicit ties are particularly valuable for addressing the cold-start problem because they can be inferred from minimal user activity. However, most existing SRS approaches focus on the user's egocentric network (direct friends) and overlook broader sociocentric structures a gap that community detection can fill [50].

III. Community Detection for Recommender Systems

1. Collaborative Filtering Enhanced by Community Structure

Sheng et al. [32] addressed the weakness of GNN-based collaborative filtering under sparse data conditions. Their method constructs a user–user network from historical user–item interaction data, then applies community detection to partition users into groups with shared interests. The discovered community memberships are used to mine users' latent attribute preferences, which are then injected into a GNN-based collaborative filtering model to produce final recommendations. The approach explicitly combines the local cohesion of communities with the high-order propagation capability of graph neural networks. Evaluated on two large real-world datasets, the proposed algorithm outperforms both standard collaborative filtering and state-of-the-art GNN-based recommendation methods, confirming that community-aware neighborhood construction substantially reduces the impact of data sparsity on recommendation quality.

Tokala et al. [33] proposed the Community-Based Matrix Factorization (CBMF) framework, which directly integrates community detection into the matrix factorization pipeline to reduce computation time while improving recommendation quality.

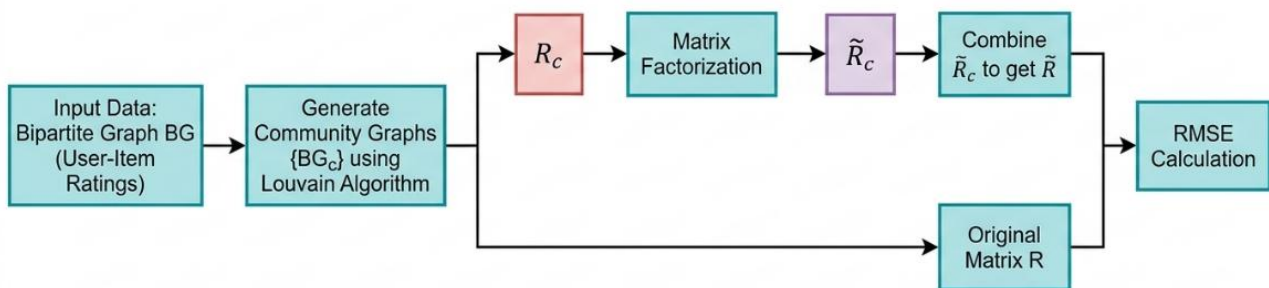


Figure 24 : Overview of the CBMF recommendation pipeline.

Rather than applying matrix factorization over the entire user-item rating network, the method first identifies communities within the rating graph using community detection algorithms, then applying matrix factorization locally within each community. This decomposition reduces the size of the matrices that need to be factorized, significantly lowering computational cost without sacrificing accuracy. The authors also incorporated the FANMF variant of non-negative matrix factorization, which accounts for user and item biases. Experiments show consistent improvements in RMSE over standard matrix factorization baselines, demonstrating that community structure in rating networks constitutes exploitable information for more efficient and accurate recommendations.

2. Addressing the Cold-Start Problem with Community Detection

Sánchez-Moreno et al. [18] tackled two well-known weaknesses in music recommender systems: the cold-start problem affecting new users with no interaction history, and the gray-sheep problem affecting users with unusual preferences. Their approach builds an implicit social network from user rating patterns and detects communities within it using structural similarity. A user similarity metric is then computed from both community membership and available ratings, capturing homophily effects between users who belong to the same community. This community-informed similarity significantly improves recommendation reliability precisely in cases where traditional collaborative filtering breaks down for new users and atypical users since community membership provides an alternative signal when rating data is scarce.

3. Combining Semantic Content and Community Structure

Rostami et al. [34] developed a hybrid recommender system for food recommendations that uses graph clustering to capture community structure among users and incorporates temporal dynamics into the similarity computation. Users are grouped via a weighted graph where edge weights reflect a time-aware similarity metric, giving more recent interactions higher influence. Community membership from this graph constrains the neighbourhood used during rating prediction, ensuring that recommendations are driven by users who are both structurally close in the network and behaviourally aligned over time.

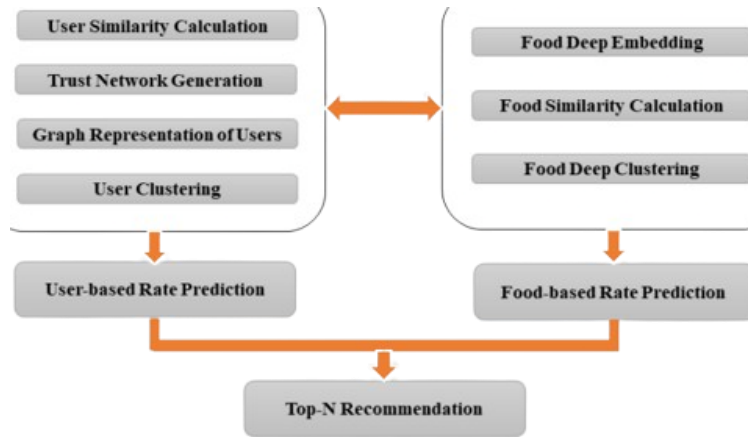


Figure 25 : Schematic representation of the hybrid food recommendation model [34].

The framework also addresses the cold-start problem by mapping new users to existing communities based on minimal available data and inheriting the community's collective rating history. Evaluated on the Allrecipes dataset using Precision, Recall, F1, AUC, and NDCG, the system outperforms existing food recommender baselines across all five metrics.

4. Community-Based Collaborative Filtering

In traditional collaborative filtering, the neighborhood set for a target user is defined by pairwise similarity scores. Under data sparsity, this approach often fails because users rarely share enough co-rated items [50].

Community-based collaborative filtering replaces the similarity-based neighborhood with a community-defined neighborhood [50]. The neighborhood consists of all users belonging to the same community as the target user. The predicted rating for an unseen item is then computed by averaging the ratings of community peers, with the implicit assumption that community members share similar preferences. This assumption has been validated empirically across multiple domains [50].

Gasparetti et al. [50] distinguish between two paradigms within this framework. In single-domain recommendation, the neighborhood consists solely of users from the community most relevant to the target user, limiting computation to a submatrix of the full user-item matrix and naturally reducing data sparsity. In multi-domain recommendation, the neighborhood extends across multiple communities to which the target user belongs, aggregating preferences from broader interest groups.

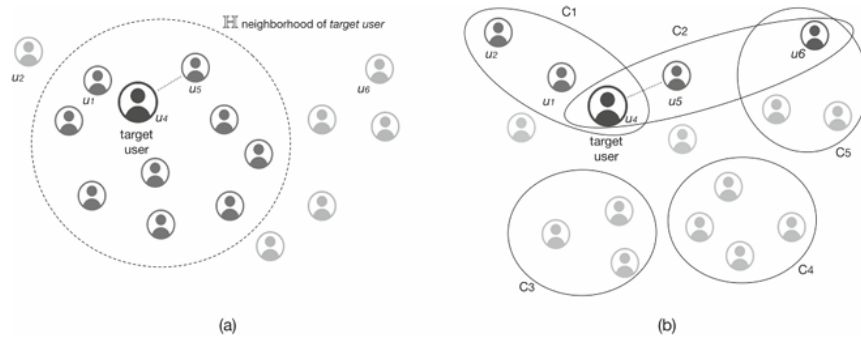


Figure 26 : Comparison between traditional neighborhood-based collaborative filtering (a) and community-based collaborative filtering (b) [50].

This distinction is particularly important for cold-start scenarios: by linking new users to an existing community based on minimal available data, the system can generate meaningful recommendations without requiring co-rated items. This approach has been validated across diverse domains including movies, music, e-commerce, and news [50].

5. Community-Aware Matrix Factorization

Matrix factorization methods have become the dominant paradigm for collaborative filtering. However, standard matrix factorization ignores community structure. To address this limitation, researchers have proposed community-aware matrix factorization formulations [50], [51].

The key idea is to add a social regularization term that encourages users within the same community to have similar latent factor vectors [50]. This means that if two users belong to the same detected community, their learned preference representations are pushed closer together during training. This regularization has been shown to improve prediction accuracy, particularly in sparse rating matrices, because community structure provides additional guidance for learning user representations when interaction data is limited [51].

A concrete instantiation of this idea is proposed by Keerthika et al. [49], who apply SVD locally within each detected community submatrix rather than globally on the full user-item matrix. By first partitioning users into communities via the Louvain algorithm and then performing independent matrix factorization within each partition, the resulting sub-matrices are denser and more homogeneous, allowing SVD to converge faster and with less computational noise. This local factorization strategy reduces the dimensionality load on the model while preserving predictive accuracy, achieving a competitive RMSE of 0.9966 on the Netflix Prize dataset. Beyond accuracy, this gives a significant interpretability advantage: recommendations can be explained through community membership rather than opaque latent factors, addressing one of the key limitations of standard matrix factorization and deep learning-based methods [49].

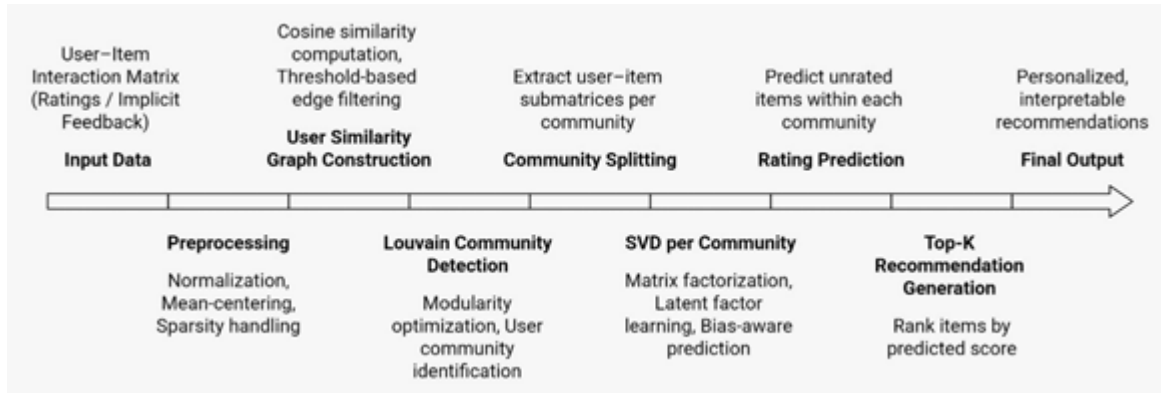


Figure 27 : Community-constrained SVD matrix factorization workflow [49].

Building on these foundations, we propose and evaluate a set of hybrid approaches that combine community detection with SVD, ALS, and BM25 scoring within an e-commerce recommendation context, aiming to assess the practical utility of different community structures beyond structural metrics alone.

IV. Proposed Hybrid Approaches

Building on the foundations of community-based collaborative filtering and community-aware matrix factorization reviewed above, we propose a set of hybrid approaches that go beyond pure community membership for recommendation. While community-based collaborative filtering constrains the candidate space to users or items within the same community, it treats all candidates equally without accounting for individual preference strength. Similarly, while community-aware matrix factorization introduces social regularization to push similar users closer in the latent space, it does not leverage the full filtering power of community structure for candidate generation. To address these limitations, we propose five hybrid approaches that combine community detection with latent factor models and retrieval algorithms. These methods use communities to constrain the candidate space while employing more sophisticated scoring mechanisms for final ranking, combining the interpretability and sparsity-reduction benefits of community structure with the fine-grained preference modeling of SVD, ALS, and BM25.

1. User-Based Approach

The User-Based approach is one of the fundamental paradigms of collaborative filtering, operating on the principle that users who have agreed in the past are likely to agree again in the future [36]. In the context of community-enhanced recommendation, this approach first constructs a user-to-user similarity network from historical rating data.

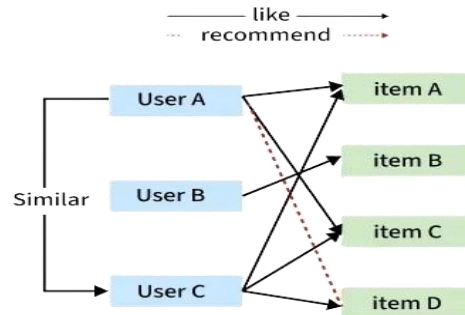


Figure 28 : User-Based Collaborative Filtering

Similarity Computation. For each pair of users, the Jaccard similarity coefficient is computed based on the items they have both rated. This metric measures the proportion of overlap between two users' rated items relative to the total unique items they have rated.

The resulting similarity matrix is symmetric, with values ranging from zero to one, where higher values indicate stronger preference overlap between users [50]. Alternative similarity measures, such as Pearson correlation and cosine similarity, have also been widely studied in the literature [2], [36].

To construct a sparse graph suitable for community detection, a threshold is applied to the similarity matrix. An edge between two users exists only if their similarity meets or exceeds this threshold. This thresholding step eliminates weak or spurious connections, producing a user graph where edges represent meaningful preference similarity [49]. This approach aligns with the methodology described by Gasparetti et al. [50], who note that implicit ties derived from co-rating patterns are particularly valuable for building reliable user graphs, as they capture genuine preference alignment rather than superficial social connections.

Once the user graph is constructed, community detection algorithms are applied to partition users into groups of individuals with shared tastes. Studies have shown that modularity-optimizing algorithms such as Louvain and Leiden are particularly effective for this task, as they maximize the density of internal connections within communities while minimizing cross-community ties [51]. For a target user, the recommendation set consists of items purchased or positively rated by other members of that user's community, excluding any items the target user has already interacted with [49], [50].

The User-Based approach offers strong personalization by leveraging the collective preferences of similar users within the same community [50]. It is particularly effective when user rating data is sufficiently dense. However, it faces challenges in extremely sparse datasets, where two users may share few or no co-rated items, making similarity computation unreliable. This limitation is one of the key motivations for exploring Item-Based alternatives [51].

2. Item-Based Approach

The Item-Based approach applies the same core principle as the User-Based method but operates on items rather than users. This approach was popularized by Sarwar et al. and has become widely adopted in commercial recommender systems due to its stability and efficiency [36], [51].

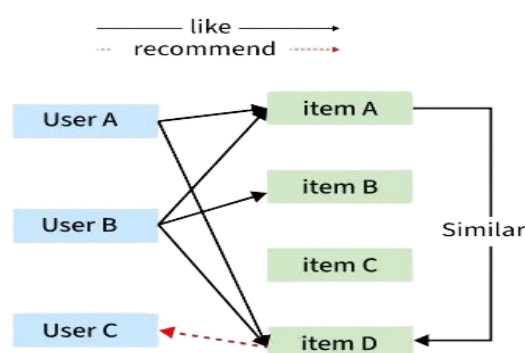


Figure 29 : Item-Based Collaborative Filtering

Similarity Computation. An item-item similarity matrix is constructed using the Jaccard coefficient based on co-rating patterns: two items are considered similar if they have been rated highly by the same users. The Jaccard value represents the proportion of users who rated both items positively relative to all users who rated either item [49]. In practice, alternative similarity measures such as cosine similarity and adjusted cosine similarity are also commonly used, with the adjusted cosine variant accounting for differences in user rating scales [36].

Graph Construction and Community Detection. After applying a threshold to obtain a sparse item graph, community detection algorithms partition items into semantically coherent groups representing product categories or thematic clusters. Items that are frequently bought or rated together by the same users naturally cluster into the same community [50]. This property is particularly valuable because it allows the system to discover latent item relationships that may not be explicitly encoded in metadata or product categories.

When a user purchases or positively rates an item, the system recommends other items belonging to the same community as that item, again excluding items the user has already consumed. As noted by Keerthika et al. [49], this community-based candidate filtering ensures that recommendations remain topically coherent with the user's demonstrated interests.

The Item-Based approach offers two significant advantages over its User-Based counterpart [51]. First, it handles cold-start users more effectively, as recommendations do not depend on the user's history beyond the triggering item. Even a single interaction can trigger meaningful recommendations. Second, it performs well in scenarios where user similarity data is sparse, such as new platforms or niche product categories, because item similarity tends to be more stable over time than user similarity [50]. Choudhary et al. [51] further note that item-based methods are computationally more efficient in online recommendation scenarios, as item similarities can be precomputed offline and updated periodically, whereas user similarities often require real-time computation.

3. Singular Value Decomposition with Community Filtering

Singular Value Decomposition (SVD) is a matrix factorization technique that decomposes the user-item rating matrix into low-dimensional latent factors [45]. Each user and item is represented as a vector in a shared latent space, where the dot product between a user vector and an item vector predicts the user's preference for that item. SVD is particularly effective at capturing latent patterns in user preferences that are not directly observable from explicit co-purchasing patterns.

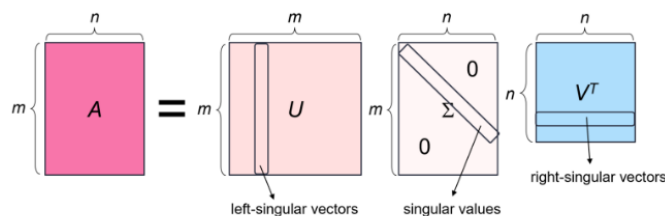


Figure 30 : SVD of a matrix

In our hybrid SVD approach, we first compute a user-user similarity graph using the Jaccard coefficient as described in the User-Based Approach. Community detection algorithms (Louvain, Leiden, Infomap, and FluidC) are applied to this graph, producing a partition where users with similar purchasing behaviors are grouped together. The SVD model is then trained on the user-item rating matrix to learn latent factor representations for all users and items.

At recommendation time, the process combines both sources of information:

1. Community-based candidate filtering: For a target user, we identify their community and collect all items purchased by other members of that community, excluding items the user has already purchased. This creates a candidate set that is topically coherent with the user's interest cluster.
2. SVD-based candidate ranking: Each candidate item is scored by computing the dot product between the user's latent factor vector and the item's latent factor vector. Items are then ranked by this predicted preference score, and the top N items are recommended.

This hybrid approach offers a compelling synergy: the community structure ensures that recommendations remain relevant to the user's broad interest area, while SVD provides fine-grained ranking within that area based on latent preference patterns that may not be captured by simple co-purchasing statistics.

4. Alternating Least Squares with Community Filtering

Alternating Least Squares (ALS) is another matrix factorization technique that is particularly well-suited for implicit feedback data [46]. Unlike SVD, which typically requires explicit ratings, ALS uses a confidence weighting scheme that treats the presence or absence of interactions as signals of varying confidence. Each observed interaction (a user purchasing an item) receives a high confidence weight, while unobserved interactions receive lower confidence weights.

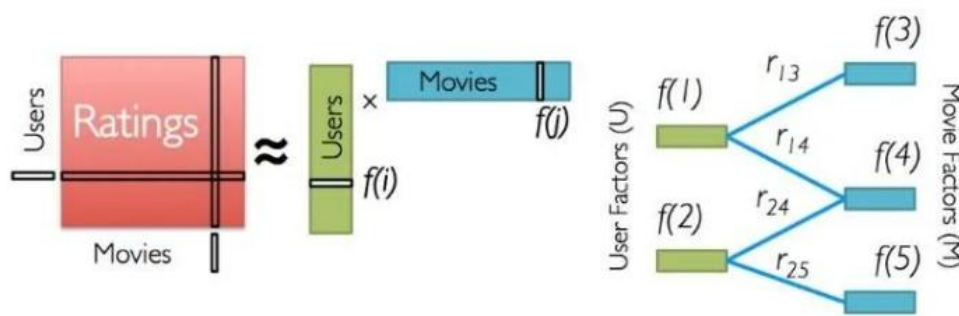


Figure 31 : Alternating Least Squares Method

Our ALS-based hybrid recommender follows the same architecture as the SVD approach. Community detection is first performed on the user-user similarity graph, partitioning users into interest-based groups. The ALS model is then trained on the user-item interaction matrix, learning latent factor vectors for users and items using the implicit feedback formulation described by Hu, Koren, and Volinsky [47].

During recommendation generation, the system:

1. Identifies the target user's community and aggregates items purchased by community peers to form a candidate set
2. Scores each candidate using the ALS-predicted preference (dot product of user and item latent vectors)
3. Returns the top N highest-scoring items not already purchased by the user

The ALS approach offers advantages in handling the inherent sparsity of implicit feedback data, as it naturally down-weights the significance of unobserved interactions while amplifying the signal from observed purchases.

5. Best Matching 25 with Community Filtering

BM25 (Best Matching 25) is a ranking function originally developed for information retrieval that has been adapted for item-based collaborative filtering [48]. In this context, each user is treated as a document containing the set of items they have purchased, and each item is treated as a term appearing in multiple user documents.

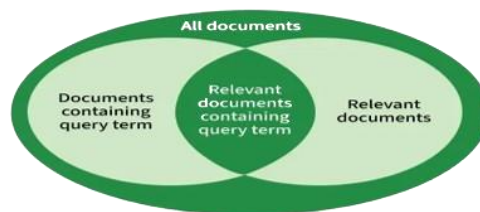


Figure 32 : Best Matching 25 Algorithm

BM25 computes the relevance of an item to a User-Based on:

- Term frequency (TF): How many times a user has purchased a given item (capped to prevent distortion)
- Inverse document frequency (IDF): How rare an item is across all users (rare items receive higher importance)
- Document length normalization: Adjusting for users who have purchased many items.

The BM25 item-item similarity matrix is constructed by computing the BM25 relevance score between every pair of items, producing a weighted graph where edge weights represent semantic or co-purchasing relatedness.

For item-based community-enhanced recommendations, we:

1. Construct an item-item graph using BM25 similarity scores rather than Jaccard similarity
2. Apply community detection algorithms to this BM25-based item graph, yielding partitions of semantically related items
3. For a target user, identify the communities containing items the user has already purchased
4. Recommend other items from these same communities, ranked by their BM25 relevance to the user's purchased items

The BM25 approach differs fundamentally from the matrix factorization methods in that it is entirely item-based and does not learn latent user representations. Instead, it leverages statistical properties of the co-purchasing matrix to identify items that are contextually relevant to a user's purchase history.

V. Conclusion of Chapter

In this chapter, we established the theoretical foundation for integrating community detection with recommender systems. We started by covering the fundamentals of recommender systems, including the three main paradigms content-based filtering, collaborative filtering, and hybrid approaches and the challenges they face, namely data sparsity, the cold-start problem, scalability, and interpretability.

We then reviewed the existing literature at the intersection of community detection and recommender systems. We looked at how researchers have used community structure to enhance collaborative filtering, address the cold-start problem, and improve matrix factorization methods.

Building on this foundation, we presented the hybrid approaches we propose in this work. These include community-based user and item approaches, as well as hybrid methods that combine community detection with SVD, ALS, and BM25 scoring. The idea behind these hybrids is to use communities to constrain the candidate space and then apply more sophisticated scoring mechanisms for final ranking, combining the interpretability and sparsity-reduction benefits of communities with the fine-grained preference modelling of latent factor models.

With this foundation in place, the next chapter presents the implementation and experimental results. We describe the experimental setup, evaluate the community detection algorithms on both real-world and synthetic benchmarks, and test the proposed hybrid recommender approaches to see which ones work in practice.

Chapter 4: Implementation and Experimental Results

Chapter 4: Implementation and Experimental Results

I. Introduction to Chapter

In this chapter we will present the complete implementation and experimental results of this research. It is organized into four main sections. First, classical community detection algorithms (Louvain, Leiden, Walktrap, Infomap, LPA, FLPA, LPA-MNI, CLPA, FluidC, Girvan-Newman, Kernighan-Lin, SpinGlass, MCL) are implemented and tested on nine real-world benchmark datasets. Second, deep learning methods (GCN, GAT, VGAE, Autoencoder) are implemented using TensorFlow/Keras and tested on the same datasets. Third, both classical and deep learning methods are evaluated on synthetic LFR benchmark networks with varying mixing parameters to assess their robustness. Finally, the best-performing models are selected and evaluated on a recommender system proxy task to assess their practical utility.

II. Experimental Setup

1. Hardware and Software Environment

Component	Specification
Processor	AMD Ryzen 5 4500U
RAM	8 GB
Operating System	Windows 11
Programming Language	Python 3.13.9
Development Environment	Jupyter Notebook

Table 1 : Hardware and Software Environment

2. Libraries and Frameworks

Library	Purpose
NetworkX	Graph manipulation
NumPy, Pandas	Numerical operations, data handling
Scikit-learn	NMI, ARI, K-means
python-louvain	Louvain algorithm
leidenalg, igraph	Leiden, Walktrap, Infomap, SpinGlass
markov-clustering	Markov Clustering (MCL)
TensorFlow/Keras	GCN, GAT, VGAE, Autoencoder

Table 2 : Libraries and Frameworks

3. Standard Benchmark Datasets for Community Detection

To fairly compare different community detection methods, researchers need standardized test networks. Synthetic benchmarks such as LFR offer controlled conditions for experimentation, but real-world networks remain essential for assessing how algorithms perform on authentic data. Over the past several decades, a collection of relatively small networks has become widely adopted as the standard test suite in this field. These datasets are popular because they are openly accessible, possess known or interpretable group structures, and span different types of real systems [11], [26].

a) Zachary's Karate Club

The Karate Club network stands as the most widely recognized dataset in social network analysis (see Figure 33). Wayne Zachary collected this data in 1977, mapping friendships among 34 members of a university karate club [38]. What makes this network particularly valuable is its natural ground truth: an actual conflict caused the club to divide into two separate factions following a disagreement between the instructor and the president. This real-world split provides an ideal reference for testing community detection algorithms. The network consists of 34 nodes connected by 78 edges. Its modest size facilitates easy visualization and rapid algorithm testing, yet its structure is sufficiently nuanced to highlight performance differences between methods [38].

According to published surveys, the Karate Club dataset appears in over 26% of community detection studies, making it the most frequently used benchmark in the literature [38], [39].

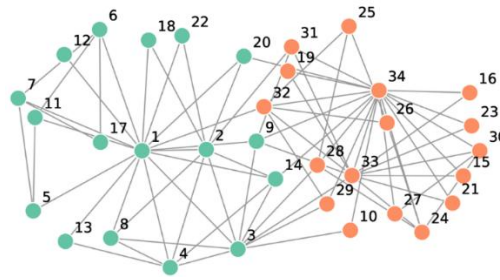


Figure 33 : Zachary's Karate Club

b) Dolphin Social Network

David Lusseau compiled the Dolphin network by observing a population of 62 bottlenose dolphins in Doubtful Sound, New Zealand [3]. Between 1995 and 2001, researchers identified each adult dolphin through natural markings on their dorsal fins and recorded which individuals were frequently seen together. An edge between two dolphins indicates that they were observed in close association more often than would be expected randomly. The network naturally separates into two primary groups, mirroring the actual social division observed in the fjord.

With 62 nodes and 159 edges, this dataset offers greater complexity than the Karate Club, including instances where some individuals may belong to multiple groups. When properly partitioned, the network yields a modularity score of approximately 0.5258, making it a useful test case for evaluating algorithmic stability and detection accuracy [26], [38].

c) American College Football Network

Michelle Girvan and Mark Newman constructed the Football network from the schedule of games played during the 2000 American college football regular season [26]. In this network, 115 nodes represent individual teams, and 613 edges represent games played between them. The ground-truth communities correspond to the 12 official conferences to which teams belong. Because teams compete more frequently against opponents within their own conference than against those from other conferences, the network exhibits a clear community structure.

This dataset is especially valuable because the true community labels are known in advance, enabling precise quantitative evaluation of detection performance. Under optimal partitioning, the Football network achieves a modularity of approximately 0.6006 [38].

d) Political Books Network (Polbooks)

V. Krebs created the Polbooks network by analyzing co-purchase relationships among books about American politics sold on Amazon around the time of the 2004 presidential election [40]. In this network, each node represents a book, and edges indicate that the same buyers frequently purchased both books together. The network displays a clear political division, with books falling naturally into three clusters corresponding to liberal, conservative, and neutral political leanings. What makes this dataset particularly interesting is that the ground-truth communities derive from semantic content (political alignment) rather than purely topological features. This allows researchers to test whether algorithms can recover meaningful semantic groupings based solely on network structure [40]. The network contains 105 nodes and 441 edges, with an average degree of 8.4 [40].

e) Primary School Contact Network (Thiers)

Stehlé and colleagues collected the Primary School network in 2009 by recording face-to-face contacts among 242 children and teachers at a school in Lyon, France [41]. The research team equipped participants with radio-frequency identification (RFID) devices that measured physical proximity at 20-second intervals over two consecutive days. The resulting network captures who interacted with whom and for how long, with edge weights representing contact duration. This dataset stands out for its high temporal resolution and for representing physical social interactions rather than virtual ones.

With 242 nodes and approximately 125,000 weighted edges, it ranks among the larger benchmark datasets commonly used in literature. The ground-truth communities correspond to school classes, since children naturally interact most frequently with classmates [41].

f) Facebook Social Network

The Facebook dataset, introduced by McAuley and Leskovec [44], consists of anonymized ego networks collected from survey participants. In network analysis, an ego-network is a subgraph centered around a single user (the "ego"), including their immediate connections (friends) and the connections among those friends. The dataset contains 4,039 nodes and 88,234 edges, with ground truth communities corresponding to user-created "circles" such as family, classmates, or coworkers [44]. This dataset is valuable for evaluating algorithms on realistic social structures with overlapping communities, where a single user naturally belongs to multiple distinct social groups.

g) LastFM Social Network

The LastFM dataset, also collected by McAuley and Leskovec [44], captures social connections and musical listening habits on the LastFM online music platform. It contains 28,281 nodes (users) and 92,752 edges (friendship links), with ground-truth communities derived from user membership in interest groups corresponding to artists, genres, or listening clubs [44]. This dataset combines social structure with explicit behavioral affinities, making it suitable for evaluating homophily driven community formation in medium-scale social graphs.

h) VS13 and VS15

The VS13 and VS15 datasets, occasionally referred to as the "versus" datasets in published work, appear in certain comparative studies of community detection methods [26]. Researchers typically employ these networks when evaluating algorithm performance across graphs with differing structural characteristics or when comparing modularity-based approaches against alternative techniques. Although less frequently used than the Karate Club or Football networks, VS13 and VS15 offer additional test cases for assessing algorithmic robustness under various network topologies [26].

i) LFR Benchmark

The Lancichinetti Fortunato Radicchi (LFR) benchmark is the most widely used standard for evaluating community detection algorithms on synthetic networks. Proposed to address the limitations of the Girvan Newman (GN) benchmark such as homogeneous degree distributions and equal-sized communities it generates more realistic graph structures [31].

LFR produces networks with properties closer to real-world systems by drawing node degrees and community sizes from power-law distributions. This results in heterogeneous structures with hubs and communities of varying sizes, better reflecting social and information networks [11], [31].

A key parameter in LFR is the mixing parameter μ , which represents the fraction of edges a node shares with nodes outside its community. Lower values of μ indicate well-defined communities, while higher values make detection more challenging. When $\mu > 0.5$, communities in the strong sense no longer exist.

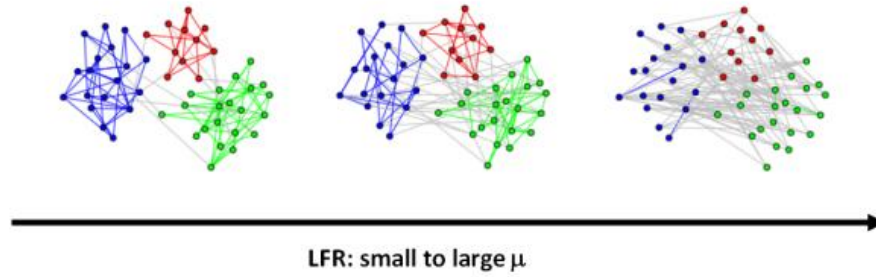


Figure 34 : Impact of the parameter μ on the detectability of clusters in heterogeneous networks.

In addition to μ , LFR includes parameters such as the number of nodes, degree distribution, and community size ranges, allowing the generation of diverse network configurations for systematic evaluation [11], [31].

Compared to GN, LFR is more challenging and better reveals algorithm limitations, which is why it is widely used in benchmarking studies. For example, Anuar et al. [26] evaluated multiple algorithms on LFR networks using metrics like NMI, ARI, and Purity, showing that performance varies significantly with μ .

LFR has been extended to support directed, weighted, and overlapping communities [31]. Despite its effectiveness, it has limitations, including scalability issues and unrealistic structures at extreme μ values [26]. Nonetheless, it remains the dominant benchmark in the field.

4. Online Retail Dataset (For Recommender System Evaluation)

For the evaluation of the proposed recommender system, the Online Retail dataset from the UCI Machine Learning Repository was employed [52]. This transactional dataset contains all transactions occurring between December 1, 2010, and December 9, 2011, for a UK-based online retailer that specializes in unique all-occasion gifts. The dataset comprises 541,909 transactions across 8 attributes, including InvoiceNo (transaction identifier), StockCode (product code), Description (product name), Quantity (units purchased), InvoiceDate (transaction timestamp), UnitPrice (price per unit in sterling), CustomerID (anonymized customer identifier), and Country. A notable characteristic of this dataset is that many customers are wholesalers, which introduces specific purchasing patterns relevant to e-commerce recommendation scenarios.

For the recommendation task, the data was preprocessed by removing canceled transactions (identified by InvoiceNo starting with 'c'), filtering out transactions with missing CustomerIDs, and aggregating the data into a user-item interaction matrix where each entry represents the total quantity purchased of a given product by a specific customer. The dataset's inherent sparsity (approximately 98.6% missing entries) and the presence of rich transactional patterns make it particularly suitable for evaluating implicit feedback recommendation methods such as ALS and SVD [49].

5. Evaluation Metrics

Two sets of evaluation metrics were used in this work, one for assessing community detection quality and one for evaluating recommender system performance.

Community Detection Metrics:

- Modularity (Q): Measures the quality of detected communities without requiring ground truth.
- Normalized Mutual Information: Measures similarity between detected communities and ground truth partitions.
- Adjusted Rand Index (ARI): Measures similarity between two clusterings adjusted for chance.

Recommender System Metrics:

- Precision@K: Measures the proportion of recommended items in the top-K list that are relevant.
- Recall@K: Measures the proportion of relevant items successfully retrieved within the top-K list.
- F1@K: The harmonic mean of Precision@K and Recall@K. In this work, F1@5 was used as the primary comparison metric.

6. LFR Synthetic Benchmark Parameters

The LFR benchmark generates networks with realistic properties: node degrees and community sizes follow power-law distributions. The mixing parameter μ (see Figure 34) controls community strength:

Parameter	Value
μ values tested	0.3, 0.5, 0.7
Network sizes (n)	500, 1000, 5000, 10000, 50000, 100000
Average degree	15-25 (scaled with size)
Minimum community size	20
Maximum community size	100

Table 3 : LFR Synthetic Benchmark Parameters

III. Experiment 1: Community Detection Results

1. Introduction

In this section, we evaluate the performance of different community detection algorithms on real-world benchmark datasets. The goal is to identify which algorithms produce high-quality communities, both in terms of structural quality and alignment with ground-truth labels. We use three complementary metrics: Normalized Mutual Information (NMI), Adjusted Rand Index (ARI), and Modularity (Q). The results here will guide our selection of algorithms for the recommender system experiments in the next section. We first present results for classical algorithms, then for deep learning methods.

2. Results of Classical Algorithms

Algorithm	Karate	Dolphins	Football	Polbooks	PS	VS13	VS15	FB	LastFM
Louvain	0.6	0.4843	0.8903	0.5603	0.7805	0.8749	0.6329	0.2473	0.6278
Leiden	0.5878	0.5524	0.8903	0.5737	0.8475	0.8749	0.6408	0.2479	0.6254
Walktrap	0.4899	0.5816	0.8874	0.5427	0.6238	0.8662	0.5718	0.2967	0.583
Infomap	0.5684	0.5373	0.9242	0.5369	0.8475	0.8749	0.598	0.2976	0.5277
LPA	0.5056	0.6596	0.8866	0.5497	0.4169	0.8345	0.4284	0.2748	0.6294
FLPA	0.6	0.6256	0.8903	0.5537	0.6307	0.8665	0.4284	0.276	0.5791
LPA-MNI	0.5111	0.588	0.9269	0.5259	0.9281	0.8662	0.5703	0.2733	0.583
CLPA	0.5925	0.5345	0.8903	0.5427	0.8072	0.8749	0.6329	0.2467	0.6307
FluidC	0.4639	0.4807	0.8443	0.4003	0.8993	0.696	0.6848	0.2238	0.457
G-Newman	0.6804	0.9085	0.4872	0.5754	0.0339	0.0435	0.0298	Slow	Slow
K-Lin	0.493	0.3819	0.7791	0.395	0.8071	0.6862	0.6169	0.068	0.3771
SpinGlass	0.5622	0.5565	0.9065	0.4781	0.8475	0.8784	0.6448	Slow	Slow
MCL	0.7324	0.4386	0.9242	0.5209	0.4237	0.4425	0.4385	0.2885	0.4616

Table 4 : NMI (Normalized Mutual Information) - Classical Algorithms

Louvain and Leiden deliver strong, consistent NMI scores across all datasets (see Table 4) both reaching 0.89 on Football and 0.87 on VS13 though Leiden doesn't always outperform Louvain, as Louvain edges ahead on Karate (0.60 vs. 0.58), suggesting that greedy optimization can sometimes find better local optima than Leiden's refinement process. Infomap excels on networks with clear flow patterns (Football: 0.92) but drops sharply on overlapping or ambiguous structures like Facebook (0.29) and LastFM (0.52). Label propagation methods are the most unstable, with LPA fluctuating wildly from 0.65 on Dolphins to just 0.27 on Facebook, though enhanced variants like LPA-MNI and CLPA improve consistency. Finally, Girvan-Newman achieves top scores on small networks (Dolphins: 0.90, Karate: 0.68) but fails entirely on larger ones due to its $O(m^2n)$ complexity.

Algorithm	Karate	Dolphins	Football	Polbooks	PS	VS13	VS15	FB	LastFM
Louvain	0.5089	0.3472	0.8069	0.6567	0.5032	0.8986	0.5292	0.1003	0.573
Leiden	0.4646	0.3809	0.8069	0.6752	0.6061	0.8986	0.5355	0.0973	0.5452
Walktrap	0.3207	0.4662	0.8154	0.6534	0.309	0.8722	0.4426	0.1134	0.482
Infomap	0.5906	0.3659	0.8967	0.6463	0.6061	0.8986	0.3606	0.0277	0.2334
LPA	0.4588	0.4815	0.7913	0.6667	0.1666	0.8427	0.252	0.0355	0.5527
FLPA	0.5089	0.4505	0.8069	0.6649	0.3396	0.8611	0.252	0.0414	0.4449
LPA-MNI	0.4809	0.4293	0.8893	0.6116	0.7994	0.8605	0.3389	0.037	0.4591
CLPA	0.4831	0.3821	0.8069	0.6487	0.5581	0.8986	0.5292	0.1068	0.5639
FluidC	0.3285	0.2799	0.6938	0.185	0.8267	0.4569	0.4626	0.0209	0.1026
G-Newman	0.7212	0.9175	0.2045	0.6795	0.0007	0.005	0.0037	Slow	Slow
K-Lin	0.3693	0.1515	0.6163	0.202	0.7011	0.4513	0.4808	0.052	0.2994
SpinGlass	0.4313	0.3792	0.8558	0.4534	0.6061	0.8986	0.5403	Slow	Slow
MCL	0.7717	0.211	0.8967	0.5936	0.1718	0.202	0.2575	0.0182	0.0407

Table 5 : ARI (Adjusted Rand Index) - Classical Algorithms

Girvan-Newman achieves a near-perfect ARI on Dolphins (see Table 5), confirming its ability to produce highly accurate partitions on small networks, but its Slow performance on Facebook and LastFM reinforce its impracticality for large-scale use. Louvain and Leiden show consistent ARI values overall, with Leiden slightly outperforming on Polbooks (0.6752 vs. 0.6567) and PS (0.6061 vs. 0.5032), suggesting that its refinement phase improves quality on more complex topologies though both perform poorly on Facebook (0.10) due to its overlapping social circles. FluidC raises concern with very low ARIs, particularly on Dolphins (0.2799) and LastFM (0.1026), reflecting its high sensitivity to initialization but it performed very well on PS (0.8267). MCL performs well on Karate (0.7717) but fails on large networks (0.0182 on Facebook), consistent with its known parameter sensitivity.

Algorithm	Karate	Dolphins	Football	Polbooks	PS	VS13	VS15	FB	LastFM
Louvain	0.4151	0.5233	0.6046	0.5272	0.3708	0.3857	0.3415	0.813	0.8154
Leiden	0.4198	0.5276	0.6046	0.5269	0.3758	0.3857	0.3417	0.8179	0.8179
Walktrap	0.3532	0.4888	0.6029	0.507	0.3375	0.3794	0.3188	0.7519	0.7727
Infomap	0.402	0.5247	0.6005	0.5268	0.3758	0.3857	0.2849	0.754	0.7349
LPA	0.4033	0.5205	0.6032	0.5154	0.2687	0.3718	0.2654	0.7607	0.7976
FLPA	0.4151	0.5194	0.6046	0.5262	0.3589	0.3742	0.2654	0.7411	0.7584
LPA-MNI	0.39	0.5258	0.601	0.5248	0.3631	0.3741	0.2932	0.7494	0.767
CLPA	0.4174	0.5177	0.6046	0.5251	0.3754	0.3857	0.3415	0.8141	0.8147
FluidC	0.3758	0.4583	0.5671	0.4108	0.2519	0.27	0.2207	0.6733	0.5651
G-Newman	0.3488	0.3815	0.4549	0.4831	0.0008	0.0001	0	Slow	Slow
K-Lin	0.384	0.4197	0.5319	0.4297	0.2825	0.2594	0.2587	0.7174	0.7263
SpinGlass	0.4158	0.5285	0.6015	0.5259	0.3758	0.3855	0.3416	Slow	Slow
MCL	0.36	0.455	0.6005	0.5138	0.2819	0.1311	0.2659	0.637	0.4126

Table 6 : Modularity - Classical Algorithms

Louvain and Leiden lead modularity, with CLPA surprisingly matching them on Facebook and LastFM (see Table 6) showing constrained label propagation can compete. Infomap trails predictably (0.75 vs. 0.81 on Facebook), since it optimizes the map equation, not modularity a useful reminder that high modularity doesn't imply flow-based structure. MCL tanks on large networks (0.63 on Facebook, 0.41 on LastFM), confirming it's ill-suited without heavy tuning.

3. Results of Deep Learning Methods

Unlike classical algorithms which operate solely on network structure, the deep learning methods particularly GCN and GAT were evaluated with node attributes derived from ground-truth labels and structural properties (e.g., role assignments for Karate Club, simulated user profiles for Facebook) to ensure a fair comparison that leverages their design for attributed graph data.

Algorithm	Karate	Dolphins	Football	Polbooks	PS	VS13	VS15	FB	LastFM
GCN	0.8372	0.8888	0.925	0.5745	0.1421	0.7871	0.6342	0.0486	0.0313
GAT	0.5756	0.8165	0.9195	0.4592	0.1119	0.8214	0.6343	0.0441	0.0295
VGAE	0.7324	1	0.9231	0.445	0.1263	0.7886	0.6095	0.0348	0.0258
AE	0.093	0.0279	0.8973	0.1635	0.1177	0.6207	0.567	0.0525	0.0296

Table 7 : Normalized Mutual Information (NMI) - Deep Learning Methods

GCN achieves near-perfect NMI on Karate, Dolphins, and Football (see Table 7), while VGAE hits a perfect 1.00 on Dolphins surpassing all classical methods on these small-to-medium networks with clear structure.

However, performance collapses on large graphs: GCN drops to 0.048 on Facebook and 0.031 on LastFM, revealing a critical limitation deep learning methods suffer from oversmoothing as network size grows, making node representations increasingly indistinguishable [15]. The autoencoder consistently underperforms across all datasets (average NMI 0.286 vs. GCN's 0.541).

Algorithm	Karate	Dolphins	Football	Polbooks	PS	VS13	VS15	FB	LastFM
GCN	0.8823	0.9348	0.8995	0.6745	0.0272	0.6808	0.4339	0.0614	0.0073
GAT	0.6682	0.8725	0.8955	0.4903	0.0129	0.7574	0.4490	0.0528	0.0064
VGAE	0.7717	1.0000	0.8905	0.4953	0.0234	0.7252	0.4248	0.0278	0.0049
AE	0.0072	0.0578	0.8358	0.0181	0.0144	0.3707	0.3719	0.0622	0.0061

Table 8 : Adjusted Rand Index (ARI) - Deep Learning Methods

VGAE achieves a perfect ARI on Dolphins (see Table 8), while GCN delivers excellent scores on Karate (0.88), Dolphins (0.93), and Football (0.89). GAT remains competitive on Football (0.89) but falls behind GCN on Karate (0.66). Yet performance collapses dramatically on large graphs, all of them drop to near-zero on LastFM, confirming that deep learning methods fail to generalize to the scale and sparsity of real social networks. The autoencoder consistently underperforms across the board (Karate: 0.0072, Dolphins: 0.0578).

Algorithm	Karate	Dolphins	Football	Polbooks	PS	VS13	VS15	FB	LastFM
GCN	0.3293	0.2682	0.0115	0.0119	0.0035	0.0147	0.0121	0.1373	0.0492
GAT	0.3192	0.3016	0.0134	0.0049	0.0012	0.0115	0.0075	0.0831	0.0486
VGAE	0.3523	0.3102	0.0188	0.0064	0.0019	0.0103	0.0171	0.0576	0.0319
AE	0.0435	0.0926	0.0008	0.0124	0.0001	0.0116	0.0135	0.0913	0.0621

Table 9 : Modularity - Deep Learning Methods

Modularity for deep learning is dismal VGAE tops out at 0.35 on Karate versus Louvain's 0.41 (see Table 9), and GCN barely hits 0.01 on Football compared to 0.60. They can match labels on small graphs, but the resulting partitions lack structural cohesion, which is exactly why they tank on larger networks.

4. LFR Benchmark Results

We move on to synthetic LFR networks here, mainly because they let us tweak things we can't control in real data specifically, the mixing parameter μ and the network size. μ basically dictates how fuzzy the communities are: 0.3 gives you nice clean clusters, while 0.5 or 0.7 makes it harder to tease apart. We also push sizes from a few hundred nodes up to 100k, because at the end of the day, we care about whether these methods can scale and stay stable when things get messy, that's what matters for practical use.

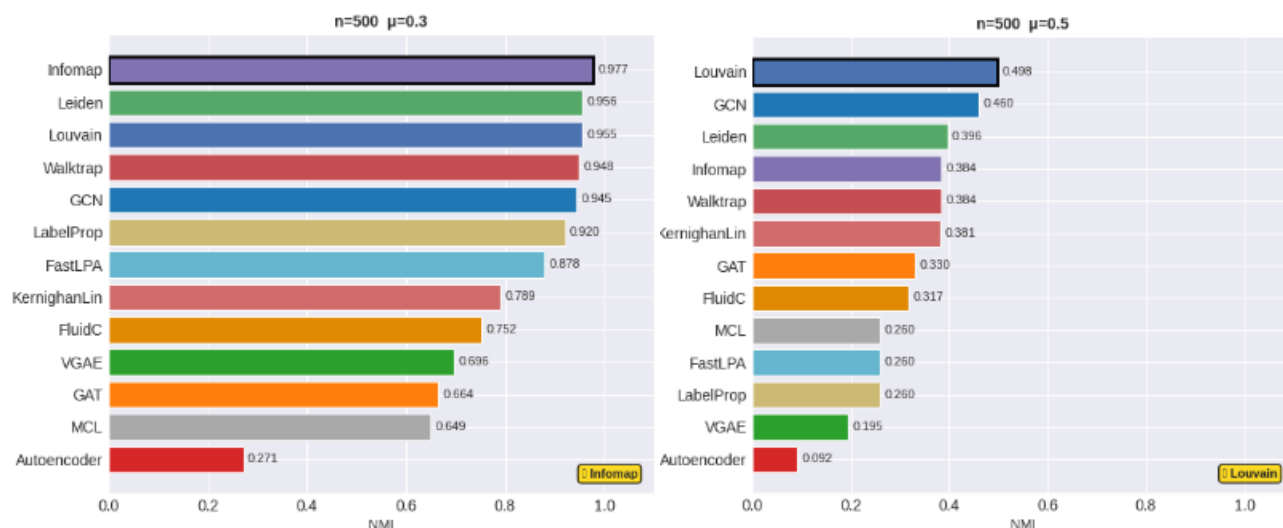


Figure 35 : LFR Benchmark Results: Network Size = 500 nodes

At $n=500$, all algorithms perform well when communities are well-defined (see Figure 35). Louvain, Leiden, Infomap, and Walktrap achieve near-perfect NMI scores (≥ 0.95). GCN and VGAE also perform competitively at $\mu=0.3$. However, as μ increases to 0.5 and 0.7, classical algorithms and GCN maintain higher accuracy while AE and VGAE drop significantly.

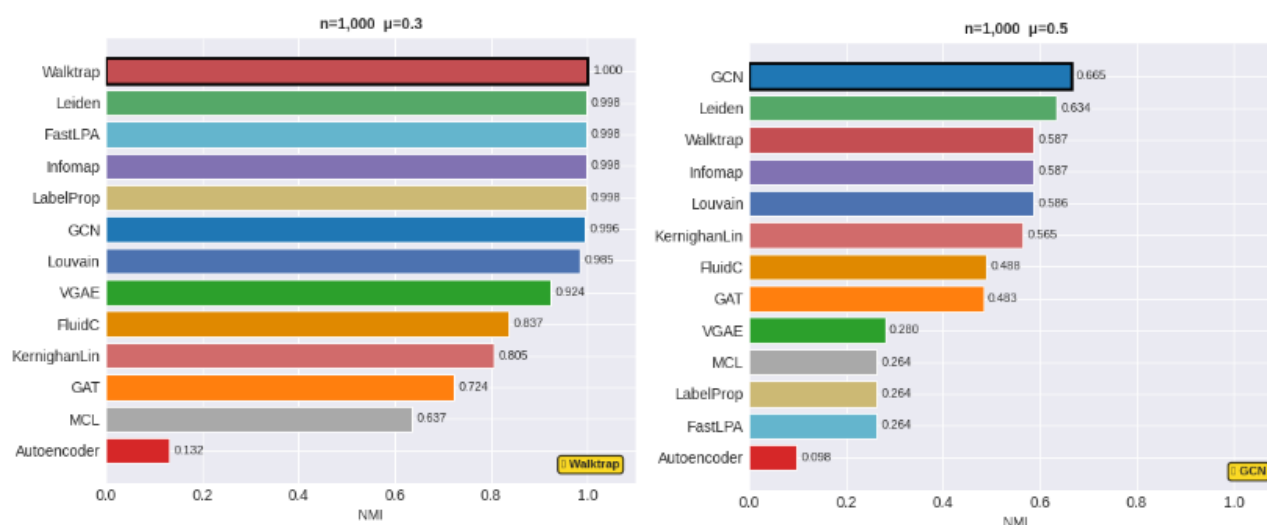


Figure 36 : LFR Benchmark Results: Network Size = 1000 nodes

At $\mu=0.3$ and 1000 nodes, pretty much everything nails it Walktrap hits perfect 1.0 (see Figure36), and Leiden, Infomap, and LabelProp are all hovering around 0.99. Nothing shocking there. Where it gets interesting is $\mu=0.5$ GCN jumps ahead of Leiden (0.66 vs. 0.63) and Louvain (0.58). That's worth noting because it hints that on networks with some fuzziness, GCN's ability to learn patterns might give it an edge over methods that just chase modularity.

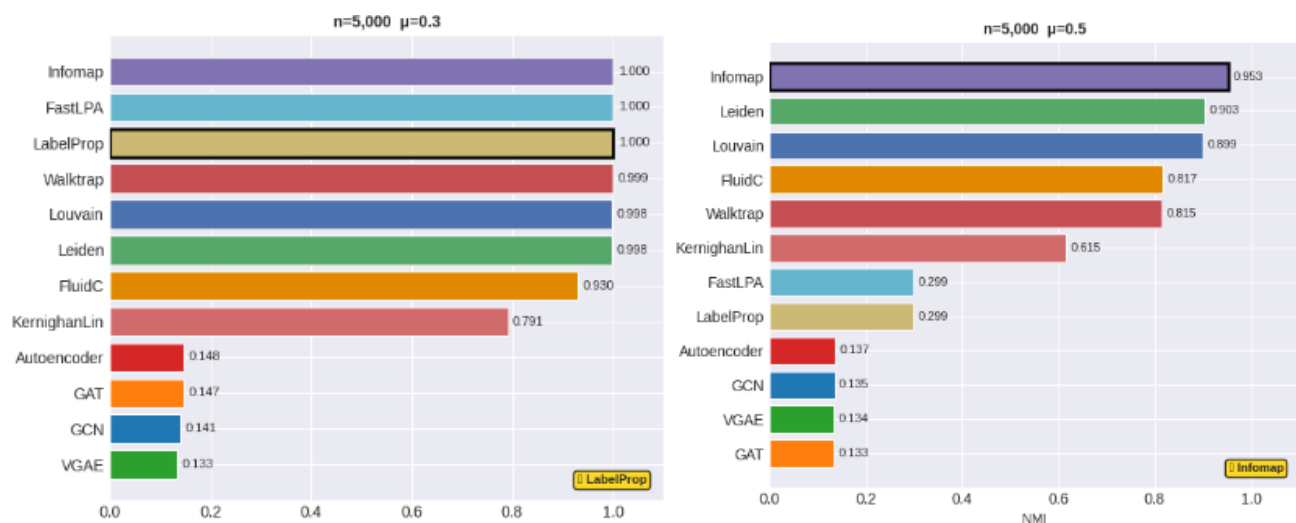


Figure 37 : LFR Benchmark Results: Network Size = 5000 nodes

At 5000 nodes and $\mu=0.3$, Infomap FastLPA and LPA hit a perfect 1.0 (see Figure 37), with Louvain Walktrap and Leiden right behind at 0.99. But GCN completely falls apart drops to 0.14, a massive drop from its performance on smaller networks. At $\mu=0.5$, Infomap stays on top (0.95 versus Leiden's 0.90 and Louvain's 0.89), while GCN keeps sinking further to 0.13. Label propagation becomes a mess at $\mu=0.5$ and above, and MCL just times out entirely at this scale. FluidC holds up reasonably well though 0.93 at $\mu=0.3$ and 0.81 at $\mu=0.5$.

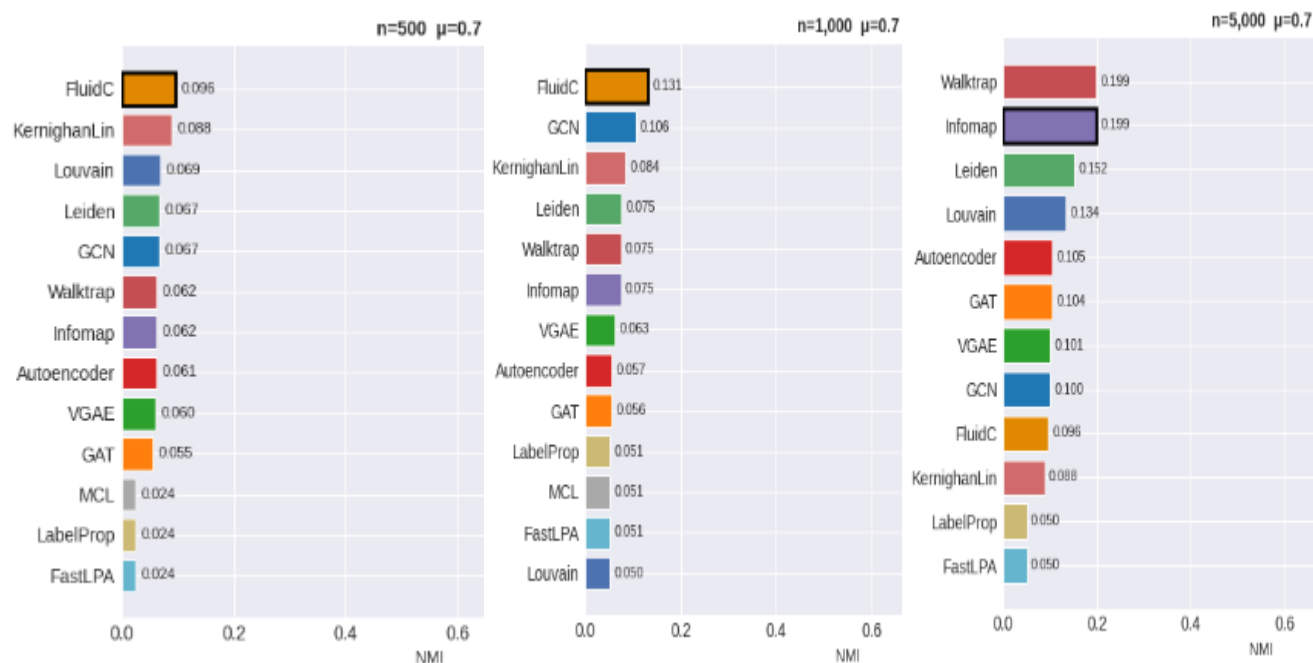


Figure 38 : LFR Benchmark Results: mixing parameter = 0.7

At $\mu=0.7$, things get messy communities are barely distinguishable (Figure 38). For $n=5000$, Walktrap and Infomap manage to lead the pack with about 0.20, while Leiden and Louvain lag a bit lower at 0.15 and 0.13, and GCN dips to 0.10. Label propagation basically falls apart, barely scraping 0.05.

The takeaway here is pretty clear: once μ goes past 0.5, strong community structure essentially disappears, and no algorithm handles it well [31].

We picked Louvain, Leiden, Infomap, and FluidC for the next phase. The selection was based on performance at $n=5000$ and $\mu=0.5$ 5000 nodes being the largest size where most algorithms could complete without timeouts, and $\mu=0.5$ offering a realistic level of ambiguity ($\mu=0.3$ is too clean, $\mu=0.7$ is nearly undetectable). At that configuration, Infomap led with 0.95, followed by Leiden at 0.90, Louvain at 0.89, and FluidC at 0.81. Deep learning methods hovered around 0.13–0.14, so they weren't competitive here. Walktrap performed reasonably (0.81) but was slower than Louvain and Leiden, and label propagation suffered from both low accuracy and high runtime, so we didn't take them forward.

For FluidC, which requires the number of communities as input, we used a square-root heuristic:

$k \approx \sqrt{n}$ clamped to the range $[2, 30]$, where n is the number of nodes in the network.

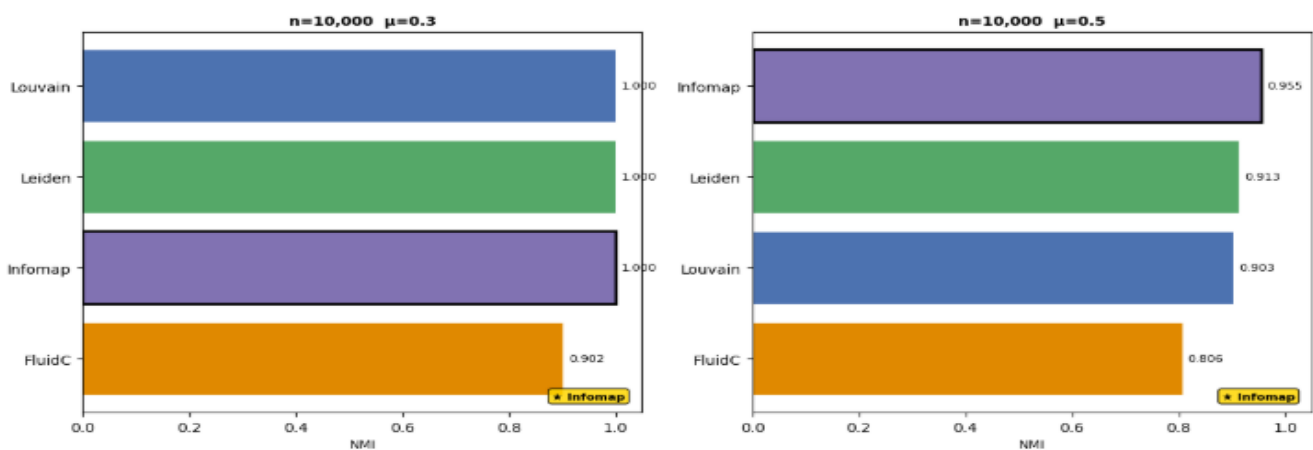


Figure 39 : LFR Benchmark Results: Network Size = 10000 nodes

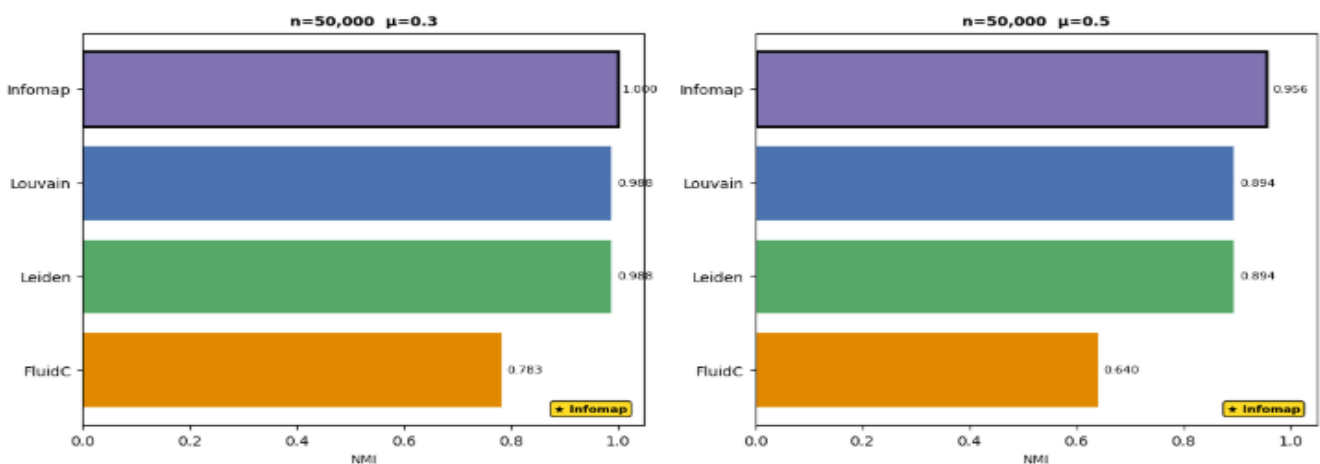


Figure 40 : LFR Benchmark Results: Network Size = 50000 nodes

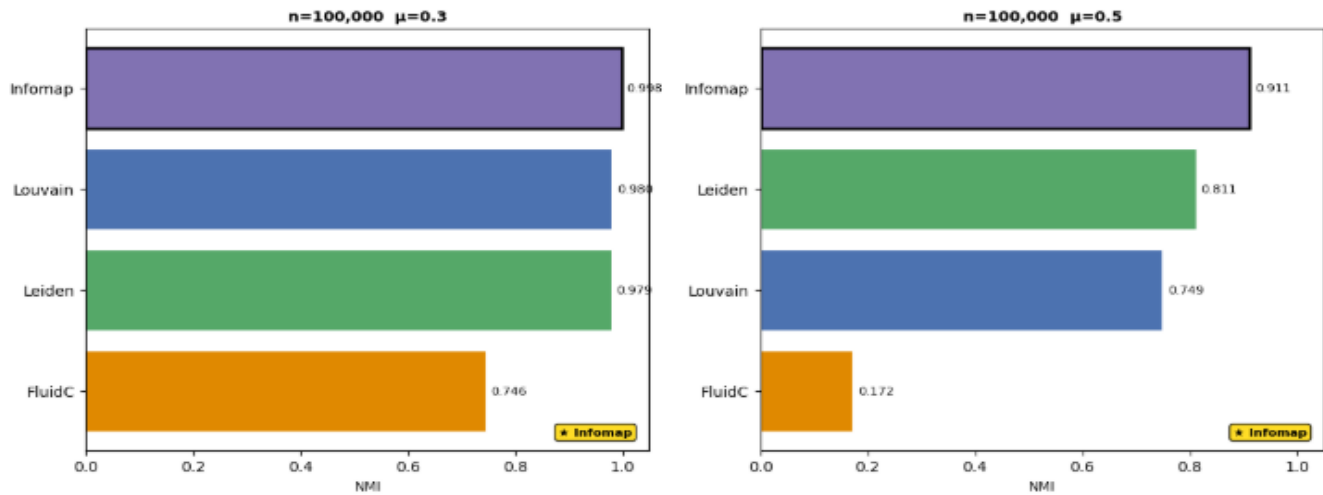


Figure 41 : LFR Benchmark Results: Network Size = 100000 nodes

On large graphs (see Figures 39, 40, 41), Louvain, Leiden, and Infomap delivered excellent accuracy at $\mu=0.3$ (NMI ~ 0.98 – 1.00) and good results at $\mu = 0.5$ (NMI ~ 0.75 – 0.96). FluidC was fast but less accurate at scale. $\mu = 0.7$ was abandoned for $n \geq 50k$ due to poor performance and excessive runtime. These four algorithms will be used in our recommender system.

IV. Experiment 2: Recommender System Integration

To evaluate the practical utility of the selected community detection algorithms (Louvain, Leiden, Infomap, and FluidC), we propose five complementary recommendation approaches that leverage detected communities for generating personalized suggestions: User-Based Collaborative Filtering, Item-Based Collaborative Filtering, SVD with Community Filtering, ALS with Community Filtering, and BM25 with Community Filtering.

1. Comparison of Recommender Systems

The following table summarizes the default and tuned configurations for ALS and BM25, along with the resulting F1@5 scores:

Model	Default Config	Default F1@5	Tuned Config	Tuned F1@5
ALS	n_factors=20, reg=0.1, $\alpha=40$, iter=10	0.1489	n_factors=100, reg=0.1, $\alpha=10$, iter=10	0.3709
BM25	k1=1.5, b=0.75	0.1782	k1=3.0, b=1.0	0.2111

Table 10 : Hyperparameter Tuning Results for ALS and BM25

We were initially puzzled by the default ALS performance (see Table 10), which was worse than the pure community baseline. After experimenting with higher latent dimensions, we found that the model was severely underfitting at rank 20. Increasing the factors to 100 allowed the algorithm to capture the necessary preference signals, which explains the sharp jump to 0.3709.

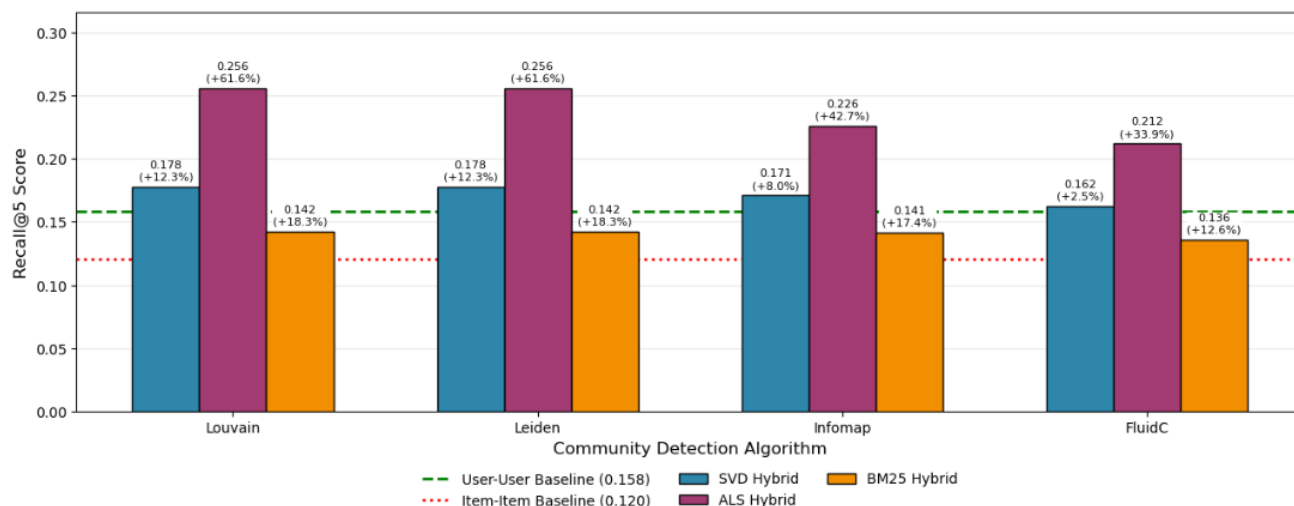


Figure 42 : Hybrid Recall@5 scores vs community baselines.

We measured Recall@5 to see how well each model retrieved products customers bought. ALS Hybrid was the clear winner (see Figure 42) at 61.6% above the User-User baseline (0.158) and stayed consistent across Louvain and Leiden. SVD Hybrid posted a decent 12.3% gain under those same structures but barely edged out the baseline at 2.5% with FluidC. BM25 Hybrid managed 18.3% over the Item-Item baseline (0.120) with Louvain/Leiden.

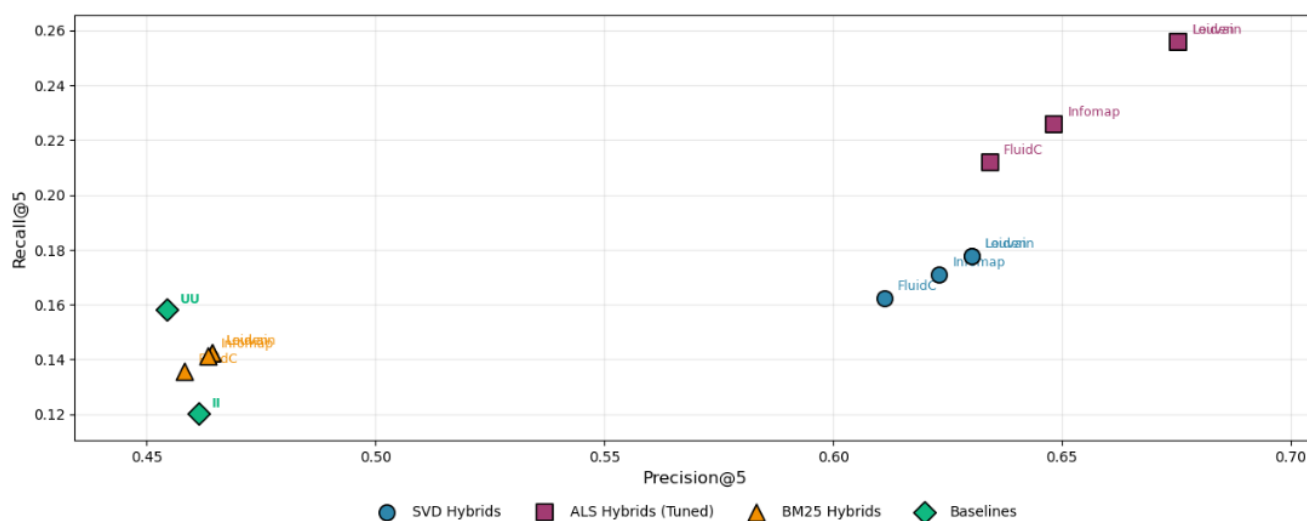


Figure 43 : Precision-recall scatter plot illustrating trade-offs across different method categories

Tuned ALS hybrids (purple squares) occupy the upper-right quadrant with the highest precision (0.63-0.68) and recall (0.21-0.26), indicating the best overall balance (see Figure 43). SVD hybrids (blue circles) show moderate precision (0.61-0.63) and lower recall (0.16-0.18). BM25 hybrids (orange triangles) and pure community baselines (green diamonds) cluster in the lower-left region with lower precision and recall.

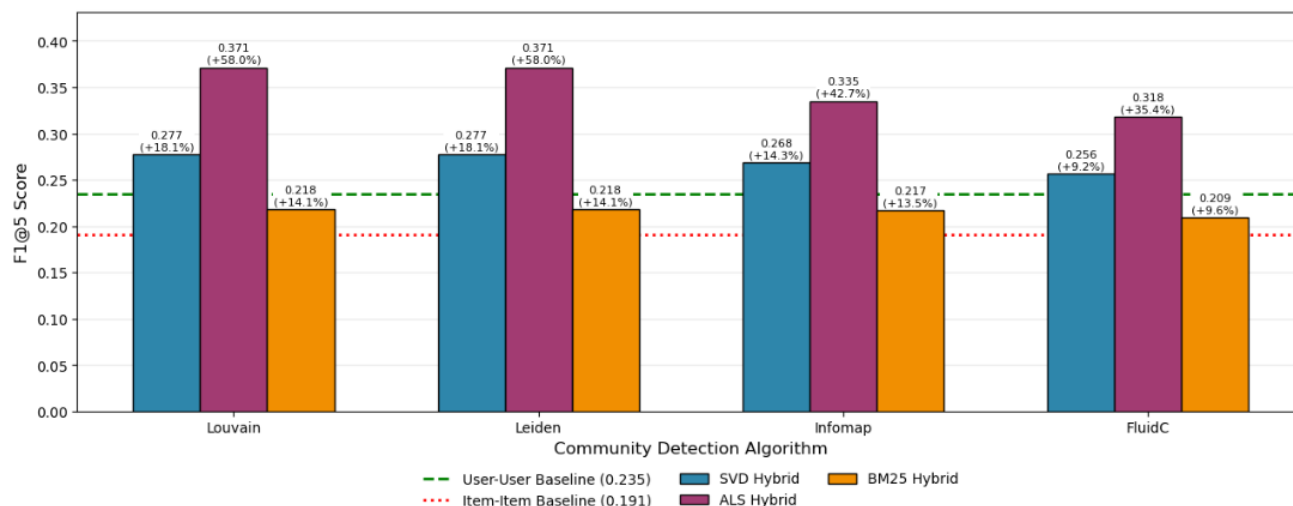


Figure 44 : Hybrid F1@5 scores vs community baselines.

F1@5 told a similar story (see Figure 44). ALS Hybrid topped out at 0.371 with Louvain or Leiden 58% above the User-User baseline of 0.235. SVD held steady with an 18% gain across both structures. BM25 managed a consistent 14% lift over Item-Item (0.191) but never beat the User-User benchmark. Bottom line: pairing ALS with Louvain or Leiden gives the best precision-recall balance.

V. Experiment 3: Web Application Interface

To make the implemented models accessible and operational, a web application was built using the Streamlit library in Python. This interactive platform allows users to configure the backend pipeline parameters and visually inspect both the community structures and the resulting personalized recommendations.

1. Configuring the Algorithm Pipeline

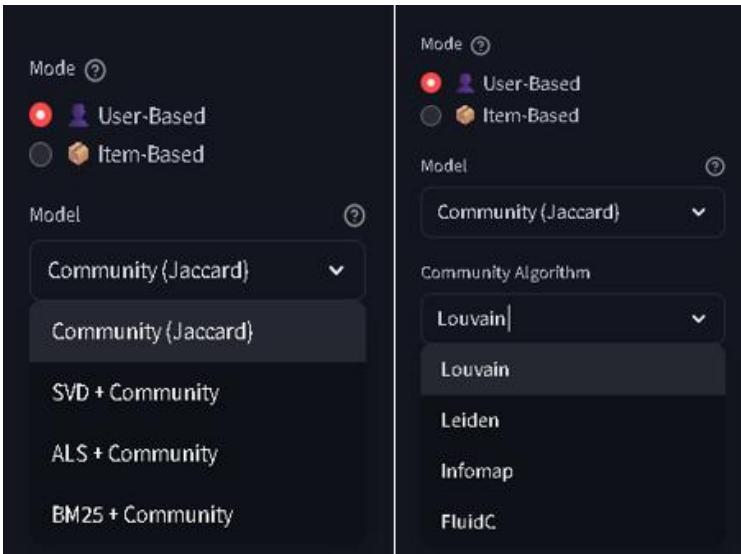


Figure 45 : Pipeline Configuration

The sidebar serves as the main control panel for configuring the recommendation pipeline before execution. As shown in Figure 45, users can select a community detection algorithm from Louvain, Leiden, Infomap, or FluidC, and pair it with a recommendation method from User-Based, Item-Based, SVD, ALS, or BM25.

2. Generating User-Based Recommendations

This interface handles the execution and output display for the user-centric filtering pathway. The workspace is structured to evaluate recommendations from a user-to-user perspective, displaying targeted client interactions and network attributes.

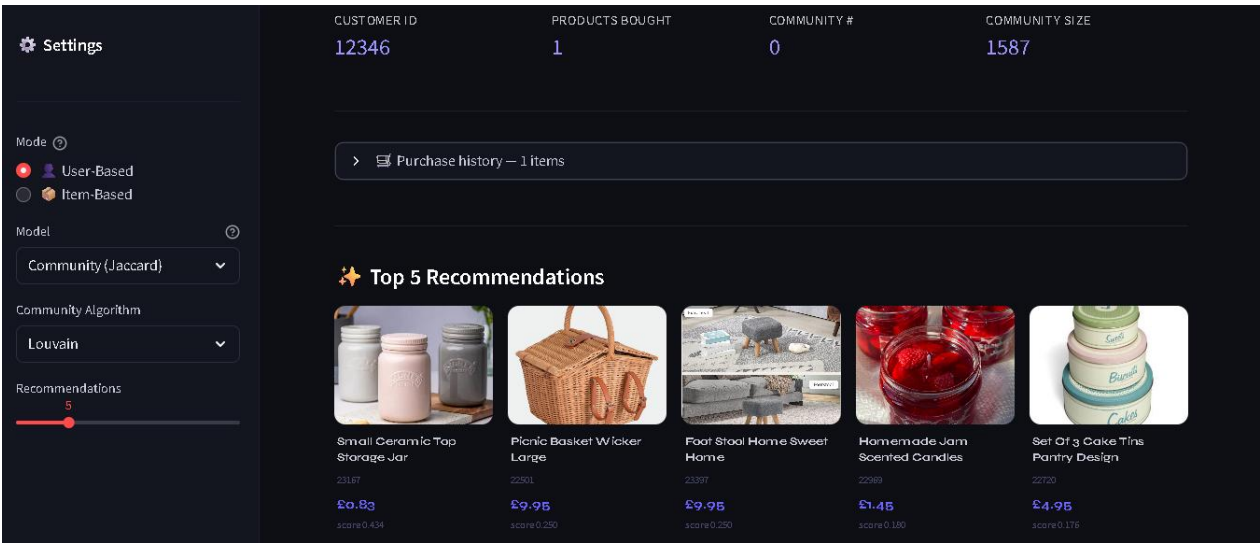


Figure 46 : User-Based Interface

When a specific user profile is selected (see Figure 46), the system identifies the user's community partition using the chosen detection algorithm and retrieves items purchased by peers within that community. The interface displays the user's purchase history alongside the generated top N recommendations, providing a transparent view of how community membership drives the suggestion process.

3. Generating Item-Based Recommendations

This view shifts the focus toward product-to-product similarities and matrix performance.

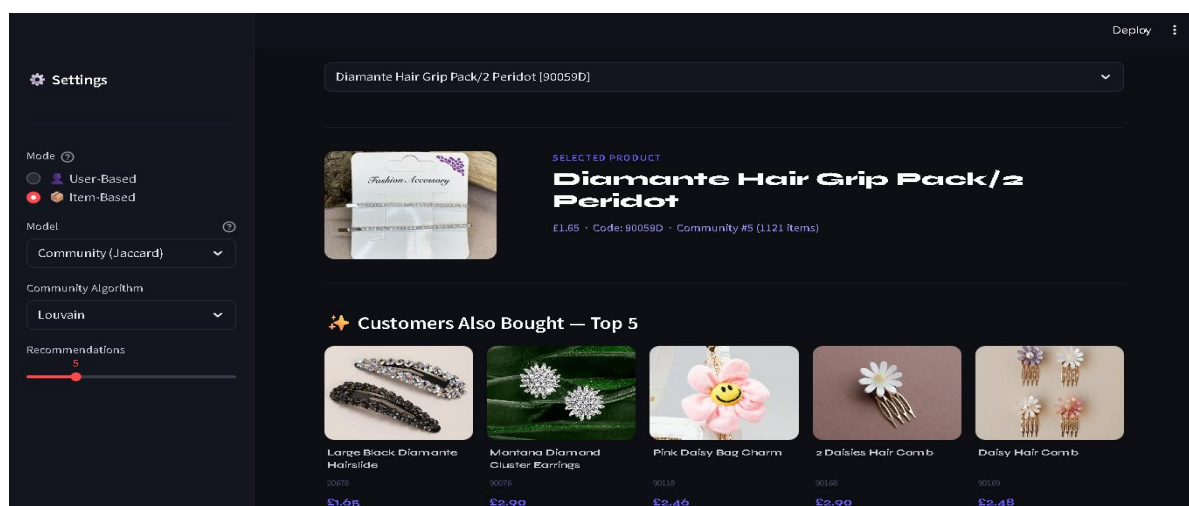


Figure 47 : Item-Based Interface

This panel provides a clean grid format containing the final recommended products (see Figure 47). Behind this screen, the system constructs an item-item similarity graph based on co-purchasing patterns, applies community detection to group semantically related products, and recommends items belonging to the same community as those already purchased by the target user.

4. Community Trending Notifications

To further support the targeted marketing objective of this work, the application incorporates a real-time community trending notification feature. As illustrated in Figure 48, when a user is browsing the platform, a Hot Deal popup notification automatically surfaces items that are currently trending within their detected community. The notification displays the product name, original and discounted price, and a community relevance indicator confirming that the suggestion is driven by collective purchasing behavior within the user's group.

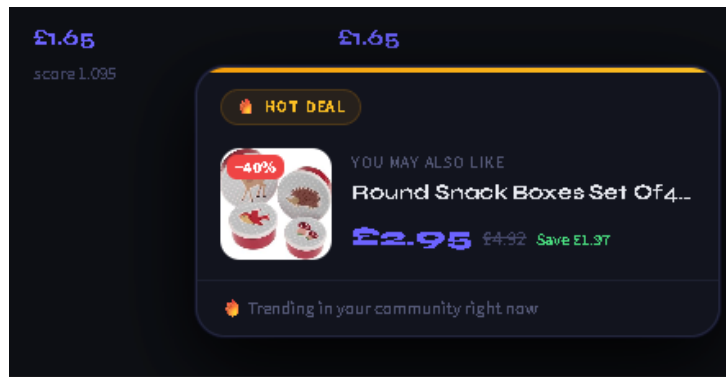


Figure 48 : Trending Community Updates

This feature bridges the gap between passive recommendation and active marketing, delivering targeted promotions directly to users based on their community membership rather than generic popularity metrics.

VI. Conclusion of Chapter

Thirteen classical algorithms and four deep learning methods were evaluated across nine real-world benchmark datasets and synthetic LFR networks of varying sizes and mixing parameters. The results consistently demonstrated that classical modularity-optimizing algorithms, particularly Louvain and Leiden, outperform deep learning approaches in both structural quality and scalability, achieving near-perfect NMI scores on well-structured networks and high modularity values exceeding 0.81 on large-scale graphs such as Facebook and LastFM. Deep learning methods, while competitive on small dense graphs like Karate Club and Football, failed to generalize to large sparse networks and degraded significantly under ambiguous community structures. Based on these findings, Louvain, Leiden, Infomap, and FluidC were selected for the recommender system phase due to their superior balance of accuracy and computational efficiency. The integration of community detection with latent factor models demonstrated substantial practical gains: the hybrid ALS+Community approach, after hyperparameter tuning, achieved an F1@5 score of 0.3709, representing a 58% improvement over the pure community baseline, while SVD-based hybrids yielded a more modest improvement of 18.1%, and BM25-based hybrids delivered a 14.1% lift against the Item-Item baseline. We take this as evidence that community structure and latent factor models capture complementary aspects of user preferences, and that combining them with proper hyperparameter optimization leads to significantly more effective recommendation. Finally, the Streamlit web application demonstrated the practical deployability of the proposed pipeline, offering User-Based and item-based recommendation interfaces alongside a community-driven trending notification feature for targeted marketing.

Chapter 5: Conclusion and Future Perspectives

Chapter 5: Conclusion and Future Perspectives

I. Summary of Contributions

Most published comparisons of community detection algorithms stop at structural metrics. We asked a different question: do the communities that score well on NMI and modularity also produce better recommendations? The answer is not straightforward and depends heavily on which scoring method sits on top of the detected communities.

We produced three main outputs. First, a systematic benchmark covering thirteen classical algorithms and four deep learning methods across nine real-world datasets and six LFR network sizes a scope wider than most individual papers in this space. Second, a set of five hybrid recommendation pipelines that pair community partitions with SVD, ALS, and BM25 scoring, evaluated end-to-end on a real ecommerce dataset. Third, a deployable Streamlit application that makes the full pipeline interactive and configurable without touching code.

The central empirical finding is that ALS+Louvain achieves $F1@5 = 0.3709$ on the UCI Online Retail dataset, a 58% gain over the community-only baseline but only after hyperparameter tuning. The default ALS configuration produces $F1@5 = 0.1489$, which is worse than the baseline.

1. Answer to the Research Question

The main research question posed in Chapter 1 was:

How do different community detection approaches, ranging from classical algorithms to modern graph learning methods, compare in terms of their effectiveness and practical utility for recommender systems?

Based on the experimental evidence presented in Chapter 4, we conclude:

Classical algorithms, particularly modularity-optimizing methods such as Louvain and Leiden, remain the most effective and practical choice for community detection in social network data. They achieve high structural quality (modularity > 0.81 on large networks), excellent scalability (near-linear time complexity), and when combined with optimally tuned ALS latent factor scoring, they yield substantial improvements in recommender system performance. Infomap is also highly competitive for ground truth recovery on smaller networks but shows slightly lower utility for recommendation tasks.

Deep learning methods (GCN, GAT, VGAE, Autoencoder) are not yet suitable for unsupervised community detection at internet scale. While they perform well on small, dense graphs (Karate Club, Football), they fail on large, sparse social networks (Facebook, LastFM) and on ambiguous community structures ($\text{LFR } \mu \geq 0.5$). Their higher computational cost and lower modularity make them impractical for real-world e-commerce or social media applications without additional node attributes or supervised signals.

The practical utility of detected communities for recommender systems depends critically on the choice of both the community detection algorithm and the scoring method, as well as hyperparameter optimization. The pure community-based approach (User-User with Louvain) provides a solid baseline ($\text{F1@5} = 0.2347$), but the hybrid ALS+Community approach dramatically improves upon it (+58.0% after tuning). The BM25-based hybrid, meanwhile, delivered a 14.1% lift against the Item-Item baseline. This is a clear sign that communities and latent factors capture complementary aspects of user preferences: communities ensure topical coherence, while ALS provides fine-grained ranking. The dramatic improvement from default ALS ($\text{F1@5} = 0.1489$) to tuned ALS ($\text{F1@5} = 0.3709$) further underscores the critical importance of hyperparameter optimization in latent factor models.

II. Future Perspectives

The following directions could extend this work:

1. Dynamic Community Detection for Temporal Recommender Systems

The communities detected till now are static snapshots of a user graph built from historical transactions. In practice, purchasing behavior shifts seasonally, and social connections evolve. A temporal extension where community membership is updated incrementally as new transactions arrive rather than recomputed from scratch would be the most direct path to making this pipeline production ready.

2. Neural Collaborative Filtering with Community Embeddings

The hybrid approach uses community membership as a hard filter for whether a user belongs to a community or does not. A softer integration would treat community membership as a continuous embedding fed as side information into a neural collaborative filtering model like LightGCN. The intuition is that community embeddings would encode group-level taste patterns that individual user embeddings, trained on sparse data, cannot reliably learn on their own.

3. Cross-Domain Evaluation

To assess generalizability, we should evaluate the best-performing methods (SVD+Louvain, SVD+Leiden) on multiple recommendation domains, including movie ratings (MovieLens), music listening (LastFM), and social media interactions (Facebook). This would reveal whether the observed gains hold across different interaction types and sparsity levels.

4. Scalable Deep Learning for Large Graphs

Recent advances in scalable GNNs (e.g., GraphSAINT, Cluster-GCN) and self-supervised learning (e.g., GraphMAE) may address the scalability limitations observed in this study.

5. Explainability and Trust

An advantage that has received little attention of community-based recommendation is explainability. Unlike latent factor models, where the reason for a recommendation is buried in a dot product between vectors no human can interpret, community-based suggestions have a natural explanation: these items are popular among buyers with similar purchase histories to yours. Whether this increases user trust has not been tested in this context and would be worth investigating.

- [1] M. E. J. Newman, *Networks: An Introduction*. Oxford, U.K.: Oxford University Press, 2010.
- [2] S. Wasserman and K. Faust, *Social Network Analysis: Methods and Applications*. Cambridge, U.K.: Cambridge University Press, 1994.
- [3] A.-L. Barabási, *Network Science*. Cambridge, U.K.: Cambridge University Press, 2016.
- [4] S. Fortunato, "Community detection in graphs," *Physics Reports*, vol. 486, no. 3–5, pp. 75–174, Feb. 2010. doi: 10.1016/j.physrep.2009.11.002.
- [5] J. Xie, S. Kelley, and B. K. Szymanski, "Overlapping community detection in networks: A survey," *ACM Computing Surveys*, vol. 45, no. 4, pp. 1–35, Aug. 2013. doi: 10.1145/2501654.2501657.
- [6] X. Huang, D. Chen, T. Ren, and D. Wang, "A survey of community detection methods in multilayer networks," *Data Mining and Knowledge Discovery*, vol. 35, no. 1, pp. 1–45, Jan. 2021. doi: 10.1007/s10618-020-00716-6.
- [7] H. Cai, V. W. Zheng, and K. C. Chang, "A comprehensive survey of graph embedding: Problems, techniques, and applications," *IEEE Transactions on Knowledge and Data Engineering*, vol. 30, no. 9, pp. 1616–1637, Sep. 2018. doi: 10.1109/TKDE.2018.2807452.
- [8] W. L. Hamilton, R. Ying, and J. Leskovec, "Representation learning on graphs: Methods and applications," *IEEE Data Engineering Bulletin*, vol. 40, no. 3, pp. 52–74, Sep. 2017.
- [9] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," *Physical Review E*, vol. 69, no. 2, p. 026113, Feb. 2004. doi: 10.1103/PhysRevE.69.026113.
- [10] M. A. Javed et al., "Community detection in networks: A multidisciplinary review," *Journal of Network and Computer Applications*, vol. 72, pp. 301–318, 2018. doi: 10.1016/j.jnca.2018.02.011.
- [11] Z. Yang, R. Algesheimer, and C. J. Tessone, "A comparative analysis of community detection algorithms on artificial networks," *Scientific Reports*, vol. 6, p. 30750, Aug. 2016. doi: 10.1038/srep30750.
- [12] G. Palla, I. Derényi, I. Farkas, and T. Vicsek, "Uncovering the overlapping community structure of complex networks in nature and society," *Nature*, vol. 435, no. 7043, pp. 814–818, Jun. 2005. doi: 10.1038/nature03607.
- [13] J. Sheykhzadeh, B. Zarei, and F. S. Gharehchopogh, "Community Detection in Social Networks Using a Local Approach Based on Node Ranking," *IEEE Access*, pp. 1–1, 2024. doi: 10.1109/ACCESS.2024.3420109.
- [14] A. Lancichinetti and S. Fortunato, "Limits of modularity maximization in community detection," *Physical Review E*, vol. 84, no. 6, p. 066122, Dec. 2011. doi: 10.1103/PhysRevE.84.066122.
- [15] J. Zhou et al., "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, 2020. doi: 10.1016/j.aiopen.2021.01.001.
- [16] D. Jin et al., "A survey of community detection approaches: From statistical modeling to deep learning," *IEEE Trans. Knowledge and Data Engineering*, vol. 35, no. 2, pp. 1149–1170, 2023. doi: 10.1109/TKDE.2021.3104155.
- [17] S. Paul, A. Chakraborty, S. Dutta, K. Das Ghosh, and A. Mitra, "Leveraging the Commerce Graph for Optimizing Digital Commerce," *Procedia Computer Science*, 2025. doi: 10.1016/j.procs.2025.04.061.

- [18] D. Sánchez-Moreno, V. F. López Batista, M. D. Muñoz Vicente, Á. L. Sánchez Lázaro, and M. N. Moreno-García, "Social network community detection to deal with gray-sheep and cold-start problems in music recommender systems," *Information*, vol. 15, no. 3, p. 138, 2024. doi: 10.3390/info15030138.
- [19] A. Kanavos, G. Vonitsanos, I. Karamitsos, and K. Al-Hussaeni, "Exploring network dynamics: Community detection and influencer analysis in multidimensional social networks," in *Proc. IEEE Int. Conf. Big Data (BigData)*, Washington, DC, USA, Dec. 2024, doi: 10.1109/BigData62323.2024.10825058.
- [20] S. K. S., A. Sreekumar, and K. Balakrishnan, "Text-attributed community detection in complex networks through LLMs and GNNs: A powerful fusion of language and graphs," *Neurocomputing*, 2025. doi: 10.1016/j.neucom.2025.130573.
- [21] A. Nasirzonouzi, "Leveraging network science for customer segmentation and product recommendation," *Northeast Journal of Complex Systems*, vol. 7, no. 1, Art. 3, 2025. doi: 10.22191/nejcs/vol7/iss1/3.
- [22] I. Saikalyan, V. Kumar, and P. Radha Krishna, "Recommender system using local community detection and non-negative matrix factorization," in *Proc. Int. Conf. Advanced Communications and Machine Intelligence (MICA 2023)*, vol. 405, Smart Innovation, Systems and Technologies, Springer, Singapore, 2024. doi: 10.1007/978-981-97-6222-4_16.
- [23] S. Hu, C. Wang, C. Qin, H. Zhu, and H. Xiong, "Super-node generation for GNN-based recommender systems: Enhancing distant node integration via graph coarsening," in *Proc. 29th Int. Conf. Database Systems for Advanced Applications (DASFAA)*, Gifu, Japan, Jul. 2024, pp. 353–363, doi: 10.1007/978-981-97-5572-1_24.
- [24] V. A. Traag, L. Waltman, and N. J. Van Eck, "From Louvain to Leiden: guaranteeing well-connected communities," *Scientific Reports*, vol. 9, no. 1, p. 5233, Mar. 2019. doi: 10.1038/s41598-019-41695-z.
- [25] K. Sliwa, E. Kušen, and M. Strembeck, "A case study comparing Twitter communities detected by the Louvain and Leiden algorithms during the 2022 war in Ukraine," in *Companion Proc. ACM Web Conf. 2024 (WWW '24 Companion)*, Singapore, May 2024, pp. 1–6. doi: 10.1145/3589335.3651892.
- [26] S. H. H. Anuar, Z. A. Abas, N. M. Yunus, M. F. Mukhtar, T. Setiadi, and A. S. Shibghatullah, "Comparison of community detection algorithm based on modularity," *Pertanika J. Sci. & Technol.*, vol. 32, no. 3, pp. 1285–1300, Apr. 2024.
- [27] A. Gates, I. B. Wood, W. P. Hetrick, and Y.-Y. Ahn, "A guide for choosing community detection algorithms in social network studies: The Question-Alignment approach," *American J. Preventive Medicine*, vol. 59, no. 4, pp. 597–605, Sep. 2020. doi: 10.1016/j.amepre.2020.04.015.
- [28] F. Papadopoulos, A. Kitsak, M. Á. Serrano, M. Boguñá, and D. Krioukov, "Geometrical inspired pre-weighting enhances Markov clustering community detection in complex networks," *Applied Network Science*, vol. 6, no. 1, p. 78, Nov. 2021. doi: 10.1007/s41109-021-00370-x.
- [29] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, New York, NY, USA, Aug. 2014, pp. 701–710. doi: 10.1145/2623330.2623732.
- [30] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 1024–1034. <https://doi.org/10.48550/arXiv.1706.02216>

- [31] A. Lancichinetti, S. Fortunato, and F. Radicchi, "Benchmark graphs for testing community detection algorithms," *Physical Review E*, vol. 78, no. 4, p. 046110, Oct. 2008. doi: 10.1103/PhysRevE.78.046110.
- [32] J. Sheng, Q. Liu, Z. Hou, and B. Wang, "A collaborative filtering recommendation algorithm based on community detection and graph neural network," *Neural Process. Lett.*, vol. 55, no. 6, pp. 7095–7112, Dec. 2023. doi: 10.1007/s11063-023-11252-x.
- [33] S. Tokala, M. K. Enduri, T. J. Lakshmi, and H. Sharma, "Community-based matrix factorization (CBMF) approach for enhancing quality of recommendations," *Entropy*, vol. 25, no. 9, p. 1360, Sep. 2023. doi: 10.3390/e25091360.
- [34] M. Rostami, M. Oussalah, and V. Farrahi, "A novel time-aware food recommender-system based on deep learning and graph clustering," *IEEE Access*, vol. 10, pp. 52508–52524, 2022. doi: 10.1109/ACCESS.2022.3175317.
- [35] F. Ricci, L. Rokach, and B. Shapira, "Recommender systems: Introduction and challenges," in *Recommender Systems Handbook*, 3rd ed. Boston, MA, USA: Springer, 2022, pp. 1-35. doi: 10.1007/978-1-0716-2197-4_1.
- [36] J. B. Schafer, D. Frankowski, J. Herlocker, and S. Sen, "Collaborative filtering recommender systems," in *The Adaptive Web*, P. Brusilovsky, A. Kobsa, and W. Nejdl, Eds. Berlin, Germany: Springer, 2007, pp. 291-324. doi: 10.1007/978-3-540-72079-9_9.
- [37] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions," *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 6, pp. 734-749, Jun. 2005. doi: 10.1109/TKDE.2005.99.
- [38] W. W. Zachary, "An information flow model for conflict and fission in small groups," *Journal of Anthropological Research*, vol. 33, no. 4, pp. 452-473, 1977. doi: 10.1086/jar.33.4.3629752.
- [39] M. Nooribakhsh, M. Fernández-Diego, F. González-Ladrón-de-Guevara, and M. Mollamotalebi, "Community detection in social networks using machine learning: A systematic mapping study," *Knowledge and Information Systems*, vol. 66, no. 12, pp. 7205-7259, Dec. 2024. doi: 10.1007/s10115-024-02201-8.
- [40] V. Krebs, "The political books network," Unpublished dataset, 2004. Available: <http://www-personal.umich.edu/~mejn/netdata/>
- [41] J. Stehlé et al., "High-resolution measurements of face-to-face contact patterns in a primary school," *PLoS ONE*, vol. 6, no. 8, p. e23176, Aug. 2011. doi: 10.1371/journal.pone.0023176.
- [42] X. Su, S. Xue, F. Liu, J. Wu, J. Yang, C. Zhou, W. Hu, C. Paris, S. Nepal, D. Jin, Q. Z. Sheng, and P. S. Yu, "A comprehensive survey on community detection with deep learning," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 4, pp. 4682-4702, Apr. 2024. doi: 10.1109/TNNLS.2021.3137396.
- [43] J. Li, S. Lai, Z. Shuai, Y. Tan, Y. Jia, M. Yu, Z. Song, X. Peng, Z. Xu, Y. Ni, H. Qiu, J. Yang, Y. Liu, and Y. Lu, "A comprehensive review of community detection in graphs," *Neurocomputing*, vol. 600, p. 128169, Oct. 2024. doi: 10.1016/j.neucom.2024.128169.
- [44] J. McAuley and J. Leskovec, "Learning to Discover Social Circles in Ego Networks," in *Advances in Neural Information Processing Systems (NIPS)*, Lake Tahoe, NV, USA, 2012, pp. 548–556.
- [45] Scikit-learn. "TruncatedSVD." Scikit-learn Documentation. Accessed: June 7, 2026. [Online]. <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.TruncatedSVD.html>

- [46] B. Frederickson. "implicit: Fast Python Collaborative Filtering for Implicit Feedback Datasets." GitHub. Accessed: June 7, 2026. [Online]. Available: <https://github.com/benfred/implicit/>
- [47] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative Filtering for Implicit Feedback Datasets," in *2008 Eighth IEEE International Conference on Data Mining, Pisa, Italy, Dec. 2008*, pp. 263-272. doi: 10.1109/ICDM.2008.22
- [48] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333-389, 2009. doi: 10.1561/15000000019
- [49] T. Keerthika, R. G. Ignisha, V. Rajamani, S. S. Kumar, K. Selvam, M. R. Aiyappan, and L. S. Thoshi Babu, "Enhancing Recommendation Systems through SVD-based collaborative filtering and community detection," *PLOS ONE*, vol. 21, no. 4, p. e0346579, Apr. 2026. doi: 10.1371/journal.pone.0346579.
- [50] F. Gasparetti, G. Sansonetti, and A. Micarelli, "Community detection in social recommender systems: a survey," *Applied Intelligence*, vol. 51, no. 6, pp. 3975–3995, Jun. 2021. doi: 10.1007/s10489-020-01962-3.
- [51] C. Choudhary, I. Singh, and M. Kumar, "Community detection algorithms for recommendation systems: techniques and metrics," *Computing*, vol. 105, no. 2, pp. 417–453, Feb. 2023. doi: 10.1007/s00607-022-01131-z.
- [52] D. Chen, "Online Retail Data Set," UCI Machine Learning Repository. Accessed: June 16, 2026. [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/Online+Retail>
- [53] M. J. Page et al., "The PRISMA 2020 statement: An updated guideline for reporting systematic reviews," *BMJ*, vol. 372, n71, Mar. 2021. doi: 10.1136/bmj.n71.
- [54] S. Jalali and C. Wohlin, "Systematic literature studies: Database searches vs. backward snowballing," in *Proc. ACM-IEEE Int. Symposium on Empirical Software Engineering and Measurement (ESEM)*, Lund, Sweden, 2012, pp. 29–38. doi: 10.1145/2372251.2372257.